Proyecto final: análisis de algoritmos en el contexto de la bibliometría

Análisis de Algoritmos

Nicolas Buitrago Martinez

1092851684

Juan David Martinez Valencia

1095208100

Luisa Fernanda Castañeda Cano

1097040431



Docente: Sergio Augusto Cardona Torres

Universidad del Quindío

Facultad de Ingeniería

Programa Ingeniería de Sistemas y Computación

Armenia

2026

Contenido

1. Introducción 4
2. Arquitectura de la aplicación 5
2.1 Componentes 5
2.2 Flujo de datos 6
2.3 API e interfaz 6
3. Corpus y preprocesamiento 8
3.1 Título, autores y año 9
3.2 Preprocesamiento y representaciones 10
4. Requerimiento 1: similitud textual 10
4.1 Caso de estudio 11
4.2 Distancia de Levenshtein 11
4.3 Alineamiento de Needleman-Wunsch 12
4.4 Coeficiente de Jaccard 14
4.5 Similitud del coseno con TF-IDF 14
4.6 Modelos de inteligencia artificial 15
4.7 Análisis crítico 16
5. Requerimiento 2: agrupamiento jerárquico 19
5.1 Preprocesamiento y matriz de distancias 19
5.2 Algoritmo aglomerativo 19
5.3 Dendrogramas 21
5.4 Evaluación interna: ¿qué criterio agrupa mejor? 23
6. Requerimiento 3: búsqueda de patrones 26
6.1 Caso de estudio 26
6.2 Knuth-Morris-Pratt 27
6.3 Boyer-Moore 28
6.4 Rabin-Karp 30
6.5 Análisis crítico: teoría frente a ejecución 31
6.6 Demostración paso a paso en la aplicación 34
7. Ranking de términos con ordenamientos no comparativos 35
7.1 Lista de frecuencias 35
7.2 Fundamentación teórica 36
7.3 Implementación 38
7.4 Medición de tiempos 40
7.5 Comportamiento al aumentar n y k 42
7.6 Efecto de la jerarquía de memoria 45
7.7 Validación del ranking 48
8. Requerimiento 4: despliegue y documentación 50
9. Conclusiones 51

10. Declaración de uso de inteligencia artificial 52
11. Referencias bibliográficas 53
Anexo A. Título, autores y año extraídos de los PDF 56
Anexo B. Código de los algoritmos de ordenamiento 58
Anexo C. Verificación independiente en Python 61

1. Introducción

Este documento presenta el diseño, la implementación y el análisis de la aplicación desarrollada como proyecto final del curso de Análisis de Algoritmos. La aplicación procesa el corpus de 20 artículos científicos sobre inteligencia artificial en la educación entregado por el docente y, a partir del abstract de cada uno, resuelve los tres requerimientos funcionales del enunciado (Universidad del Quindío, 2026): el análisis de similitud textual con cuatro algoritmos clásicos y dos modelos de inteligencia artificial, el agrupamiento jerárquico de los abstracts con tres criterios de enlace y la búsqueda exacta de patrones con los algoritmos de Knuth-Morris-Pratt, Boyer-Moore y Rabin-Karp. El cuarto requerimiento pide desplegar la aplicación y documentarla. Además, el capítulo 7 integra el ranking de términos con algoritmos de ordenamiento no comparativos del seguimiento, con las correcciones que pidió el docente.

Todos los algoritmos se implementaron desde cero en Java, sin funciones de alto nivel que los resuelvan: la búsqueda de patrones no usa indexOf, contains ni expresiones regulares, y las matrices de programación dinámica, los vectores TF-IDF, las fusiones del agrupamiento y las tablas auxiliares de la búsqueda se construyen con arreglos, listas y tablas hash. Los modelos de inteligencia artificial preentrenados se usan, como lo permite el enunciado, solo para convertir cada abstract en un vector; la similitud entre los vectores la calcula la aplicación.

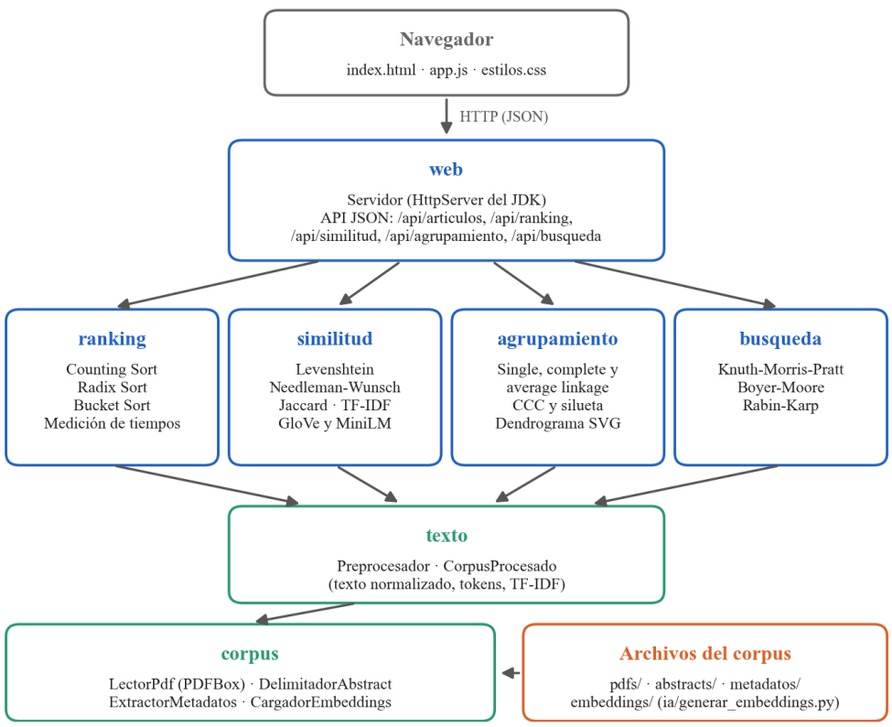
Para cada requerimiento presentamos un caso de estudio con la demostración paso a paso de cada algoritmo y un análisis crítico que contrasta la teoría con lo observado al ejecutar las implementaciones. Todas las cifras salen de la propia aplicación: el modo trazas genera las demostraciones y el modo experimentos las mediciones (la interfaz permite, además, repetir paso a paso la demostración de la búsqueda), y los archivos resultantes están en la carpeta resultados del repositorio. Las mediciones se hicieron en un portátil con procesador AMD Ryzen 7 5700U, Java 21 y Windows 11.

2. Arquitectura de la aplicación

La aplicación es un proyecto Maven (Java 17) organizado en capas (Figura 1). La interfaz web, escrita en HTML, CSS y JavaScript sin frameworks, consulta una API JSON que expone el servidor HTTP incluido en el JDK. Cada requerimiento es un paquete independiente que trabaja sobre las representaciones de los abstracts que prepara el paquete texto, y este, a su vez, depende del paquete corpus, que lee los PDF con Apache PDFBox (Apache Software Foundation, 2025). La única pieza fuera de Java es el script de Python que genera los embeddings de los modelos de IA.

Figura 1

Arquitectura de la aplicación



Nota. Las flechas indican dependencia: cada componente usa el que está debajo. Elaboración propia.

2.1 Componentes

La Tabla 1 resume la responsabilidad de cada paquete. Las clases de un mismo requerimiento comparten una interfaz (AlgoritmoSimilitud, AlgoritmoBusqueda o AlgoritmoOrdenamiento), de modo que la API, los experimentos y las pruebas las tratan de la misma forma y agregar un algoritmo no exige cambiar el resto del código.

Tabla 1

Paquetes de la aplicación

Paquete	Clases principales	Responsabilidad
corpus	LectorPdf, DelimitadorAbstract, ExtractorMetadatos, CargadorCorpus, CargadorEmbeddings	Leer los PDF, extraer abstract, título, autores y año, y cargar los embeddings
texto	Preprocesador, CorpusProcesado	Normalización, términos sin stopwords y vectores TF-IDF
ranking	RankingTerminos, ContadorFrecuencias, CountingSort, RadixSort, BucketSort, MedidorTiempos	Ranking de términos con ordenamientos no comparativos (capítulo 7)
similitud	Levenshtein, NeedlemanWunsch, Jaccard, CosenoTfIdf, CosenoEmbeddings	Requerimiento 1
agrupamiento	AgrupamientoJerarquico, EvaluacionInterna, Dendrograma	Requerimiento 2
busqueda	KnuthMorrisPratt, BoyerMoore, RabinKarp, BusquedaCorpus, PasoAPaso	Requerimiento 3
web	Servidor, Json	API HTTP y archivos de la interfaz
experimentos	Experimentos, Trazas	Mediciones y demostraciones paso a paso de este documento
ia (Python)	generar_embeddings.py	Embeddings de GloVe y de all-MiniLM-L6-v2
verificacion (Python)	verificar_conteo.py	Verificación independiente del conteo y del ranking

2.2 Flujo de datos

Al arrancar, la aplicación carga el corpus una sola vez: lee los 20 PDF, extrae el abstract, el título y los autores de cada uno y los guarda en archivos de texto que reutiliza en las ejecuciones siguientes, carga los embeddings y calcula las representaciones que necesitan los algoritmos (sección 3.2). Cada consulta de la interfaz solo ejecuta los algoritmos sobre esas representaciones, así que la respuesta llega en milisegundos, salvo la matriz de Levenshtein, que es el algoritmo más costoso (sección 4.7).

2.3 API e interfaz

La Tabla 2 describe las ocho consultas de la API. La interfaz tiene una pestaña para el corpus, otra para el ranking de términos y una por cada requerimiento (Figura 2): en la de similitud se eligen dos o más artículos y se obtienen las matrices de los seis algoritmos y, al comparar dos, la matriz paso a paso de Levenshtein o de Needleman-Wunsch (sección 4.3);

en la de agrupamiento se elige la representación y se ven las métricas, los tres dendrogramas y los artículos de cada grupo; y en la de búsqueda se escribe un patrón y se ven sus ocurrencias resaltadas, las estadísticas de los tres algoritmos y su recorrido paso a paso (sección 6.6), o se escriben varios patrones separados por comas para comparar Rabin-Karp multipatrón con KMP y Boyer-Moore ejecutados una vez por patrón (sección 6.5).

El título, los autores y el año extraídos de los PDF se cruzan con los resultados de cada requerimiento: identifican los artículos comparados en la pestaña de similitud, los miembros de cada grupo al cortar los dendrogramas y los abstracts en los que aparece el patrón buscado, de modo que cada resultado se puede leer en términos de qué artículos y qué autores lo producen.

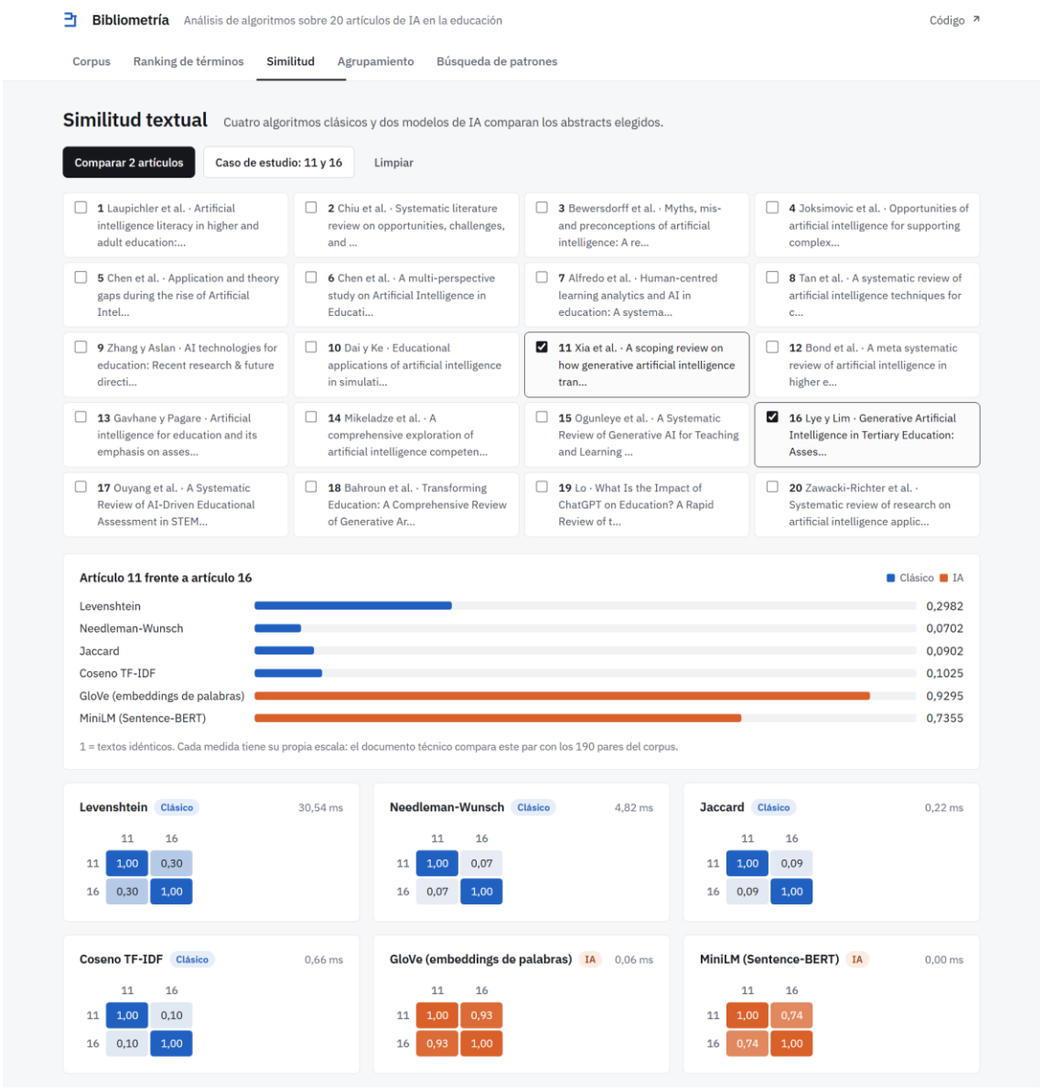
Tabla 2

Consultas de la API

Consulta	Parámetros	Respuesta
GET /api/articulos	Ninguno	Número, título, autores, año, palabras y abstract de los 20 artículos
GET /api/ranking	Ninguno	Cifras del corpus, top 15 de Counting, Radix y Bucket Sort, coincidencia de los tres rankings y tiempos
GET /api/similitud	articulos=11,16 (dos o más)	Matriz de similitud y tiempo de cada uno de los seis algoritmos
GET /api/matriz	algoritmo=levenshtein o needleman, a=11, b=16, n=12 (de 2 a 20)	Matriz de programación dinámica de los primeros n caracteres o términos, camino óptimo y costos de cada operación
GET /api/agrupamiento	representacion=tfidf, minilm o glove	Fusiones, CCC, silueta para k = 2 a 8, grupos del mejor corte y dendrograma SVG de cada criterio
GET /api/busqueda	patron=learning, mayusculas=si o no	Ocurrencias con su contexto y comparaciones, alineaciones, colisiones y tiempos de cada algoritmo
GET /api/busqueda-multiple	patrones=learning,analysis (de 2 a 10), mayusculas=si o no	Ocurrencias de cada patrón y comparaciones, alineaciones, colisiones y tiempos de Rabin-Karp multipatrón frente a KMP, Boyer-Moore y Rabin-Karp una vez por patrón
GET /api/pasos	patron=analysis, articulo=18, posicion=426, q=pequeno o grande	Tablas auxiliares y cada alineación de KMP, Boyer-Moore y Rabin-Karp en la oración de esa ocurrencia

Figura 2

Interfaz web: comparación de los artículos del caso de estudio



Nota. Captura de la aplicación en ejecución. Los tiempos de la interfaz corresponden a una sola consulta, sin calentamiento de la JVM; los de la sección 4.7 son la mediana de varias repeticiones.

3. Corpus y preprocesamiento

El corpus son los mismos 20 artículos del seguimiento, publicados entre 2019 y 2024 por cinco editoriales. El abstract de cada uno se extrae con el mismo módulo, que reconoce los encabezados de las cinco editoriales, detecta el final del abstract (en el artículo 9, por el cambio de diseño de la página) y repara las palabras partidas al final de línea usando el texto completo del artículo como vocabulario. Los 20 abstracts suman 4.007 palabras.

3.1 Título, autores y año

El enunciado pide extraer también el título y los autores. Cada editorial guarda estos datos de forma distinta, así que la clase ExtractorMetadatos combina tres fuentes y conserva la que da más información: los metadatos XMP del PDF (la lista dc:creator, completa en Elsevier y en parte de Springer), el diccionario Info del PDF (campos Title y Author) y la propia primera página. Para la primera página, PDFBox entrega cada palabra con su tamaño de letra: el título son las palabras escritas con la letra más grande y los autores, la línea que sigue al título. Se descartan los títulos de metadatos que no son títulos reales, como «ET-04-2023-0117_proof» en el artículo de Emerald, y se tolera una línea intermedia en letra pequeña, porque en ese mismo artículo el encabezado de margen («AI for education») queda en la misma línea visual que el título.

La Tabla 3 muestra qué fuente resolvió cada caso. Con estas reglas se obtuvieron el título y la lista completa de autores de los 20 artículos (Anexo A), que verificamos contra la primera página de cada PDF. Como con los abstracts, el resultado queda en archivos de texto revisables (carpeta corpus/metadatos) que el programa regenera si se borran.

Tabla 3

Fuente del título y de los autores según la editorial

Editorial	Artículos	Título	Autores
Elsevier	1 a 10	Diccionario Info	Metadatos XMP
Springer	11, 12, 17, 20	Diccionario Info	XMP en 11 y 17; primera página en 12 y 20, porque Info solo trae el primer autor
Emerald	13	Primera página (letra más grande)	Primera página
Wiley	14	Primera página (letra más grande)	Primera página
MDPI	15, 16, 18, 19	Diccionario Info	Diccionario Info

El año de publicación se toma de la línea de cita de la revista en la primera página, que cada editorial escribe a su manera: «Computers and Education: Artificial Intelligence 3 (2022) 100101» en Elsevier, «(2024) 21:40» en Springer, «Educ. Sci. 2024, 14, 636.» en MDPI, «Eur J Educ. 2024;59:e12663» en Wiley y «Vol. 66 No. 6, 2024» en Emerald. Si

ninguna aparece se usa el año del aviso de derechos de autor, pero la cita tiene prioridad porque da el año del volumen: el artículo 2 (Chiu et al.) se publicó en línea en diciembre de 2022, con «© 2022», y pertenece al volumen de 2023. Con esta regla se obtuvo el año correcto de los 20 artículos, que queda como tercera línea del archivo de metadatos de cada uno.

3.2 Preprocesamiento y representaciones

Sobre cada abstract se aplican las reglas del seguimiento: normalización Unicode NFKD, minúsculas, eliminación de tildes, tokenización por caracteres que no son letras y eliminación de 263 stopwords en inglés, 92 en español y de los tokens de una letra. A partir de ahí se construyen tres representaciones, cada una adecuada para un tipo de algoritmo:

- Texto normalizado (solo letras y espacios), entrada de Levenshtein, que compara carácter por carácter.
- Secuencia de términos sin stopwords, entrada de Needleman-Wunsch (que tiene en cuenta el orden) y de Jaccard (que la usa como conjunto).
- Vector TF-IDF (Salton y Buckley, 1988; Manning et al., 2008). El peso del término t en el documento d es $w(t, d) = tf(t, d) \cdot idf(t)$, con $tf = f(t, d)/|d|$, la frecuencia relativa del término en el abstract, e $idf = \ln(N/df(t))$, donde $N = 20$ y $df(t)$ es el número de abstracts que contienen t. Un término presente en los 20 abstracts, como «artificial» o «intelligence», tiene $idf = 0$ y no aporta a la similitud.

4. Requerimiento 1: similitud textual

La aplicación compara dos o más abstracts con seis algoritmos. Los cuatro clásicos combinan, como sugiere el enunciado, dos enfoques: programación dinámica sobre secuencias (Levenshtein y Needleman-Wunsch) y modelos de conjuntos o de espacio vectorial (Jaccard y coseno TF-IDF). Los dos de inteligencia artificial representan cada abstract con un embedding, un vector denso generado por un modelo preentrenado, y comparan los vectores con la similitud del coseno.

4.1 Caso de estudio

Elegimos los artículos 11 (Xia et al., 2024) y 16 (Lye y Lim, 2024) porque tratan el mismo tema con palabras distintas: los dos estudian cómo la inteligencia artificial generativa obliga a repensar la evaluación en la educación superior. El primero es una revisión de alcance de la literatura; el segundo propone principios para rediseñar la evaluación ante herramientas como ChatGPT. Un lector los consideraría claramente relacionados aunque compartan pocas palabras, lo que los hace un buen caso para contrastar los enfoques.

4.2 Distancia de Levenshtein

La distancia de Levenshtein (1966) es el número mínimo de inserciones, eliminaciones y sustituciones de caracteres necesarias para convertir una cadena en otra. Se calcula con programación dinámica: D[i][j] es la distancia entre los primeros i caracteres de a y los primeros j de b (Cormen et al., 2022).

$$D[i][0] = i, \quad D[0][j] = j$$

$$D[i][j] = \text{mín}(\, D[i-1][j] + 1,\ D[i][j-1] + 1,\ D[i-1][j-1] + [a_i \neq b_j] \,)$$

Como cada abstract tiene más de mil caracteres, la matriz completa tiene más de 1,4 millones de celdas; para mostrar el llenado tomamos la primera palabra de contenido de cada abstract: «generative» (artículo 11) y «emergence» (artículo 16). La Tabla 4 muestra la matriz. Cada celda toma el mínimo de tres caminos: la celda de arriba más uno (eliminar el carácter de la fila), la de la izquierda más uno (insertar el de la columna) y la diagonal más cero o uno, según los caracteres coincidan. Por ejemplo, D[2][1] = 1 porque «e» = «e» y se copia la diagonal D[1][0] = 1 sin costo, mientras que D[1][1] = 1 sale de sustituir «g» por «e». La distancia es la última celda, D[10][9] = 6, y la similitud normalizada es 1 − 6/10 = 0,40.

Tabla 4

Matriz de programación dinámica de Levenshtein para «generative» y «emergence»

	ε	**e**	**m**	**e**	**r**	**g**	**e**	**n**	**c**	**e**
ε	0	1	2	3	4	5	6	7	8	9
g	1	1	2	3	4	4	5	6	7	8
e	2	1	2	2	3	4	4	5	6	7
n	3	2	2	3	3	4	5	4	5	6
e	4	3	3	2	3	4	4	5	5	5
r	5	4	4	3	2	3	4	5	6	6
a	6	5	5	4	3	3	4	5	6	7
t	7	6	6	5	4	4	4	5	6	7
i	8	7	7	6	5	5	5	5	6	7
v	9	8	8	7	6	6	6	6	6	7
e	10	9	9	8	7	7	6	7	7	6

Nota. Filas: «generative»; columnas: «emergence»; ε es la cadena vacía. La celda inferior derecha es la distancia.

Con los abstracts completos (1.338 y 1.072 caracteres) el algoritmo llena 1.436.747 celdas y obtiene una distancia de 939, es decir, una similitud de $1 - 939/1.338 = 0{,}2982$. El costo es $\Theta(m \cdot n)$ en tiempo; en espacio la implementación guarda solo dos filas, $\Theta(n)$, porque para obtener la distancia no hace falta reconstruir las operaciones.

4.3 Alineamiento de Needleman-Wunsch

Needleman-Wunsch (Needleman y Wunsch, 1970) calcula el alineamiento global óptimo de dos secuencias. Lo aplicamos a las secuencias de términos sin stopwords, con +1 por coincidencia, −1 por sustitución y −1 por hueco:

$$F[i][0] = -i, \quad F[0][j] = -j$$

$$F[i][j] = \max(F[i-1][j-1] + s(a_i, b_j),\ F[i-1][j] - 1,\ F[i][j-1] - 1)$$

La Tabla 5 muestra la matriz para los ocho primeros términos de cada abstract. Cada celda guarda el mejor puntaje del alineamiento parcial; por ejemplo, F[1][2] = 0 porque «generative» = «generative» y se suma +1 a F[0][1] = −1. Al final, el recorrido inverso desde F[8][8] reconstruye el alineamiento óptimo:

```
Artículo 11: - | generative | artificial | intelligence | provides | opportunities | challenges
| higher | education
```

```
Artículo 16: emergence | generative | artificial | intelligence | - | ai | chatgpt | sparked |
significant
```

Es decir, un hueco frente a «emergence», luego «generative artificial intelligence» alineado con sus iguales, un hueco frente a «provides» y cuatro sustituciones. El puntaje es $3 - 6 = -3$ y hay 3 coincidencias en 9 posiciones, una identidad de $3/9 = 0{,}33$.

Tabla 5

Matriz de Needleman-Wunsch para los ocho primeros términos de cada abstract

	–	emer.	gene.	arti.	inte.	ai	chat.	spar.	sign.
–	0	−1	−2	−3	−4	−5	−6	−7	−8
gener.	−1	−1	0	−1	−2	−3	−4	−5	−6
artif.	−2	−2	−1	1	0	−1	−2	−3	−4
intel.	−3	−3	−2	0	2	1	0	−1	−2
provi.	−4	−4	−3	−1	1	1	0	−1	−2
oppor.	−5	−5	−4	−2	0	0	0	−1	−2
chall.	−6	−6	−5	−3	−1	−1	−1	−1	−2
higher	−7	−7	−6	−4	−2	−2	−2	−2	−2
educa.	−8	−8	−7	−5	−3	−3	−3	−3	−3

Nota. Filas: artículo 11 (gener. = generative, artif. = artificial, intel. = intelligence, oppor. = opportunities, chall. = challenges, educa. = education); columnas: artículo 16 (emer. = emergence, gene. = generative, arti. = artificial, inte. = intelligence, chat. = chatgpt, spar. = sparked, sign. = significant).

Con los abstracts completos (112 y 89 términos) el alineamiento tiene 114 posiciones y solo 8 coincidencias: identidad de 0,0702 y puntaje −98. El costo es $\Theta(m \cdot n)$ en tiempo y en espacio, porque aquí sí se guarda la matriz completa para reconstruir el alineamiento. La medida es exigente: solo cuenta las palabras iguales que aparecen en el mismo orden relativo en los dos textos.

La pestaña de similitud permite seguir este cálculo en la aplicación con la vista Matriz paso a paso. Al comparar dos artículos, la aplicación llena celda por celda la matriz de Levenshtein, sobre los primeros n caracteres del texto normalizado, o la de Needleman-Wunsch, sobre los primeros n términos (n entre 2 y 20): resalta la celda que se calcula y las tres de las que depende, escribe la recurrencia con sus valores y, al final, marca el camino óptimo y el alineamiento que le corresponde. La matriz y el camino salen de los métodos matriz y camino de cada clase, y las pruebas unitarias verifican en 400 casos aleatorios que

la última celda es el resultado del algoritmo y que el camino suma exactamente ese valor. Con los artículos 11 y 16 y n = 12, Levenshtein da una distancia de 10 (3 coincidencias, 8 sustituciones, 1 eliminación y 1 inserción) y Needleman-Wunsch un puntaje de −7.

4.4 Coeficiente de Jaccard

El coeficiente de Jaccard (1901) mide el solapamiento entre los conjuntos de términos de los dos abstracts, sin tener en cuenta cuántas veces aparece cada término ni en qué orden:

$$J(A, B) = |A \cap B| / |A \cup B| = |A \cap B| / (|A| + |B| - |A \cap B|)$$

El artículo 11 tiene 80 términos distintos y el 16 tiene 65; comparten 12 (ai, artificial, assessment, challenges, education, generative, integrity, intelligence, provide, rethink, retrieved y students), de modo que la unión tiene 80 + 65 − 12 = 133 términos y J = 12/133 = 0,0902. Con tablas hash, construir los conjuntos cuesta O(m + n) y contar la intersección, recorriendo el conjunto más pequeño y consultando el otro, O(mín(|A|, |B|)) en promedio.

4.5 Similitud del coseno con TF-IDF

Cada abstract se representa como un vector en el espacio de los 950 términos distintos del corpus, con los pesos TF-IDF de la sección 3.2, y la similitud es el coseno del ángulo entre los dos vectores:

$$\cos\theta = (A \cdot B) / (\|A\| \, \|B\|) = \Sigma \, w_{t11} \cdot w_{t16} / (\sqrt{\Sigma} \, w_{t11}^2 \cdot \sqrt{\Sigma} \, w_{t16}^2)$$

Solo los términos comunes aportan al producto punto (Tabla 6). El que más pesa es «generative»: aparece 2 veces entre los 112 términos del artículo 11 y 7 veces entre los 89 del 16, y está en 5 de los 20 abstracts, así que idf = ln(20/5) = 1,3863 y su aporte es 0,02476 × 0,10903 = 0,0026992. En cambio, «artificial» e «intelligence» están en los 20 abstracts, tienen idf = 0 y no aportan nada aunque sean comunes a los dos textos. El producto punto vale 0,005838, las normas 0,200642 y 0,283978, y el coseno 0,005838 / (0,200642 × 0,283978) = 0,1025, que corresponde a un ángulo de 84,1°.

Tabla 6

Pesos TF-IDF de los términos comunes a los artículos 11 y 16

Término	f_{11}	f_{16}	df	idf	w_{11}	w_{16}	$w_{11} \cdot w_{16}$
generative	2	7	5	1,3863	0,02476	0,10903	0,0026992
assessment	8	3	10	0,6931	0,04951	0,02336	0,0011568
retrieved	1	1	2	2,3026	0,02056	0,02587	0,0005319
integrity	1	1	2	2,3026	0,02056	0,02587	0,0005319
rethink	1	1	2	2,3026	0,02056	0,02587	0,0005319
students	2	1	8	0,9163	0,01636	0,01030	0,0001685
challenges	1	1	6	1,2040	0,01075	0,01353	0,0001454
provide	1	1	10	0,6931	0,00619	0,00779	0,0000482
ai	2	7	18	0,1054	0,00188	0,00829	0,0000156
education	4	2	18	0,1054	0,00376	0,00237	0,0000089
artificial	2	1	20	0,0000	0,00000	0,00000	0,0000000
intelligence	2	1	20	0,0000	0,00000	0,00000	0,0000000

Nota. f: apariciones del término en cada abstract (112 y 89 términos en total); df: abstracts del corpus que lo contienen. Los términos están ordenados por su aporte al producto punto.

Los vectores son dispersos: cada abstract usa entre 65 y 80 de los 950 términos, y el producto punto se calcula recorriendo los términos del vector más corto y consultando el otro en una tabla hash, en O(mín(|A|, |B|)).

4.6 Modelos de inteligencia artificial

Los dos modelos producen un vector denso por abstract. Ambos son preentrenados: no se entrenaron con nuestro corpus sino con grandes colecciones de texto en inglés. El script ia/generar_embeddings.py los ejecuta y guarda los vectores en corpus/embeddings; la aplicación Java lee esos vectores y calcula la similitud del coseno y la distancia euclidiana, con la misma fórmula de la sección 4.5 pero sobre vectores densos, en O(d), con d la dimensión.

- **GloVe** (Pennington et al., 2014), en su versión de 100 dimensiones entrenada con Wikipedia y Gigaword (400.000 palabras). Es un modelo de embeddings de palabras, de la misma familia que Word2Vec y FastText: cada palabra tiene un único vector, aprendido a partir de las veces que aparece junto a otras. El vector del abstract es el

promedio de los vectores de sus términos; 2.341 de los 2.407 términos del corpus (97,3 %) están en el vocabulario del modelo.

- **all-MiniLM-L6-v2** (Reimers y Gurevych, 2019; Wang et al., 2020), un transformador de 6 capas entrenado para que textos de significado parecido tengan vectores cercanos. Produce un vector de 384 dimensiones para el texto y, a diferencia de GloVe, interpreta cada palabra según su contexto. Como el modelo lee como máximo 256 tokens y 13 de los 20 abstracts son más largos, cada abstract se divide en fragmentos de oraciones completas de hasta 256 tokens y se promedian sus vectores, ponderados por el número de tokens, para no perder el final del texto. Se ejecuta con FastEmbed (Qdrant, 2024) sobre ONNX Runtime.

Para el caso de estudio, el coseno es 7,4102 / (3,1913 × 2,4982) = 0,9295 con GloVe y 0,7355 con MiniLM (Tabla 7). Los dos modelos dan valores mucho más altos que los algoritmos clásicos, pero, como se discute en la sección siguiente, el valor absoluto no basta para interpretarlos.

Tabla 7

Embeddings del caso de estudio y similitud resultante

Medida	GloVe	MiniLM
Dimensión d	100	384
Primeras componentes, artículo 11	-0,0591; 0,2591; -0,0523	-0,0713; 0,0167; 0,0047
Primeras componentes, artículo 16	-0,0778; 0,2706; 0,1969	-0,0985; -0,0041; 0,0579
Producto punto A · B	7,4102	0,7355
Normas ‖A‖ y ‖B‖	3,1913 y 2,4982	1,0000 y 1,0000
Similitud del coseno	0,9295	0,7355
Ángulo	21,6°	42,6°
Distancia euclidiana	1,2668	0,7273

Nota. Los vectores de MiniLM están normalizados (norma 1), así que su coseno es directamente el producto punto.

4.7 Análisis crítico

La Tabla 8 reúne los resultados del caso de estudio. Para interpretarlos no basta el valor absoluto, porque cada medida tiene su propia escala; por eso calculamos además la similitud de los 190 pares posibles del corpus y ubicamos el par 11–16 entre ellos (Figura 3).

Tabla 8

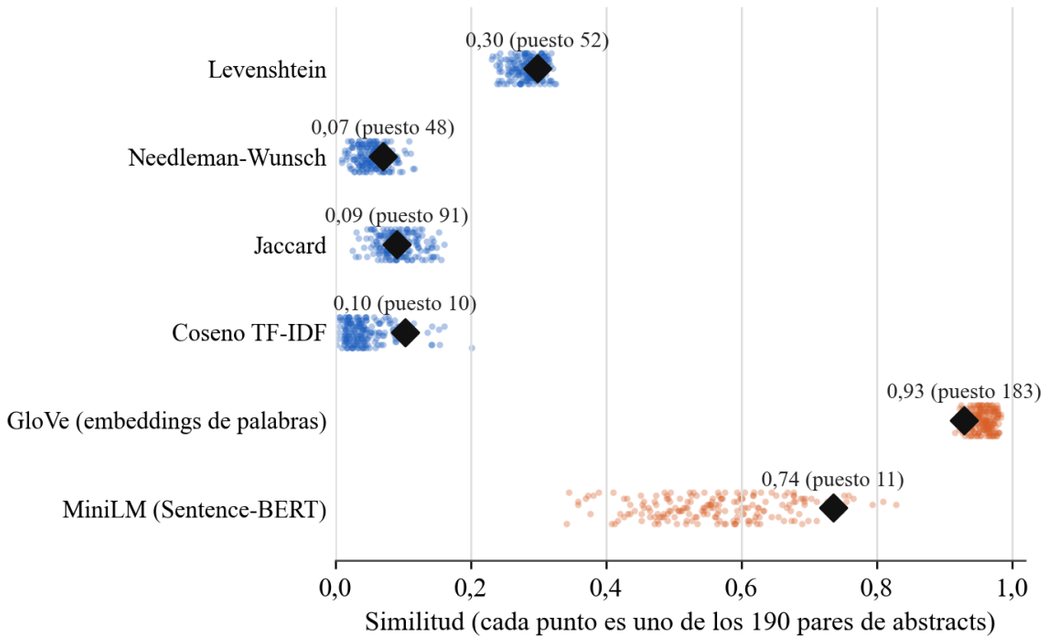
Resultados del caso de estudio frente a los 190 pares del corpus

Algoritmo	Enfoque	Valor 11–16	Puesto entre 190	Rango en el corpus	Tiempo 190 pares (ms)	Costo por par
Levenshtein	Clásico (PD)	0,2982	52	0,23 – 0,33	1.284,4	Θ(m·n), unos 1.300 caracteres
Needleman-Wunsch	Clásico (PD)	0,0702	48	0,01 – 0,12	17,62	Θ(m·n), unos 100 términos
Jaccard	Clásico (conjuntos)	0,0902	91	0,03 – 0,16	1,39	O(m + n)
Coseno TF-IDF	Clásico (vectorial)	0,1025	10	0,00 – 0,20	0,445	O(mín(|A|, |B|))
GloVe	IA	0,9295	183	0,92 – 0,98	0,043	O(d), d = 100
MiniLM	IA	0,7355	11	0,34 – 0,83	0,094	O(d), d = 384

Nota. Puesto 1 = el par más similar según ese algoritmo; PD = programación dinámica. Tiempos: mediana de 15 repeticiones tras 5 de calentamiento; en los modelos de IA no incluyen el cálculo de los embeddings, que se hace una sola vez.

Figura 3

Similitud de los 190 pares de abstracts según cada algoritmo



Nota. Cada punto es un par de abstracts; el rombo negro es el par 11–16 y entre paréntesis está su puesto (1 = el par más similar). En azul los algoritmos clásicos y en naranja los modelos de IA. Elaboración propia.

La comparación deja tres conclusiones. Primera, las medidas que solo miran las palabras ven poco en común entre los dos artículos: Jaccard y Needleman-Wunsch los ubican

en la mitad de la distribución (puestos 91 y 48), y Levenshtein casi no distingue pares, porque entre dos textos en inglés de longitud parecida la distancia de edición siempre queda entre 0,23 y 0,33: mide sobre todo la longitud de los textos y las letras frecuentes del idioma, no el tema. Segunda, el coseno TF-IDF sí reconoce el parecido (puesto 10) aunque su valor sea bajo, 0,10, porque el idf premia justo las palabras que los dos comparten y que son raras en el corpus, como «generative», «integrity» y «rethink». Tercera, entre los modelos de IA, MiniLM es el que mejor refleja que los dos artículos tratan el mismo tema (0,74, puesto 11), mientras que el 0,93 de GloVe es engañoso: al promediar los vectores de todas las palabras, todos los abstracts, que hablan de IA y educación, quedan casi en la misma dirección (todos los pares entre 0,92 y 0,98) y el par 11–16 queda de hecho en el puesto 183.

La Tabla 9 confirma estas relaciones con la correlación de rangos de Spearman (1904) entre los rankings de los 190 pares que produce cada algoritmo. Las correlaciones son moderadas: la más alta es 0,61, entre Jaccard y el coseno TF-IDF, los dos basados en términos compartidos. MiniLM se parece más al coseno TF-IDF (0,57) y a GloVe (0,53) que a las medidas de edición, y Levenshtein es el más aislado (entre 0,10 y 0,38), lo que coincide con que capture sobre todo rasgos superficiales del texto.

Tabla 9

Correlación de Spearman entre los rankings de los 190 pares

	Levensh.	N-W	Jaccard	TF-IDF	GloVe	MiniLM
Levenshtein	1,00	0,38	0,10	0,24	0,26	0,28
Needleman-Wunsch	0,38	1,00	0,41	0,44	0,37	0,35
Jaccard	0,10	0,41	1,00	0,61	0,48	0,34
Coseno TF-IDF	0,24	0,44	0,61	1,00	0,47	0,57
GloVe	0,26	0,37	0,48	0,47	1,00	0,53
MiniLM	0,28	0,35	0,34	0,57	0,53	1,00

Nota. 1 indica que los dos algoritmos ordenan los pares exactamente igual y 0 que no hay relación.

En cuanto al costo, las medidas de programación dinámica son las más caras: Levenshtein tardó 1,28 s en los 190 pares (6,8 ms por par), porque llena más de un millón de celdas por par, y Needleman-Wunsch 17,6 ms, unas 73 veces menos, porque trabaja con unos cien términos en lugar de 1.300 caracteres. Jaccard y el coseno TF-IDF tardan alrededor de

un milisegundo y los embeddings menos de una décima, porque cada par es un producto punto de 100 o 384 componentes; el costo de los modelos de IA está en generar los vectores, lo que se hace una sola vez, fuera de la aplicación. En síntesis, para comparar abstracts por tema el coseno TF-IDF es la mejor opción clásica y MiniLM la mejor en general; Levenshtein y Needleman-Wunsch son más útiles para textos cortos o casi iguales, por ejemplo para detectar copias o versiones de un mismo resumen, que para comparar contenidos.

5. Requerimiento 2: agrupamiento jerárquico

El agrupamiento jerárquico construye un árbol, el dendrograma, que muestra cómo se van uniendo los abstracts según su similitud. Implementamos desde cero la versión aglomerativa con tres criterios de enlace y la aplicamos a los 20 abstracts.

5.1 Preprocesamiento y matriz de distancias

Cada abstract se transforma con el preprocesamiento de la sección 3.2 y se representa con su vector TF-IDF; la distancia entre dos abstracts es $d = 1 - \cos\theta$, que vale 0 para textos con la misma distribución de términos y 1 para textos sin términos informativos en común. La aplicación permite cambiar la representación por los embeddings de MiniLM o de GloVe, lo que usamos en la sección 5.4 para comparar el efecto de la representación.

5.2 Algoritmo aglomerativo

El algoritmo parte de 20 grupos de un artículo y repite $n - 1 = 19$ veces dos pasos: busca en la matriz el par de grupos más cercano y los fusiona, y después actualiza la distancia del nuevo grupo a los demás. Esa actualización es lo único que cambia entre criterios y se expresa con la fórmula de Lance y Williams (1967):

- **Single linkage** (enlace simple): $d(A \cup B, R) = \min(d(A, R), d(B, R))$, la distancia entre los dos elementos más cercanos de los grupos.
- **Complete linkage** (enlace completo): $d(A \cup B, R) = \max(d(A, R), d(B, R))$, la distancia entre los dos elementos más lejanos.

- **Average linkage** (enlace promedio o UPGMA): d(A∪B, R) = (|A| d(A, R) + |B| d(B, R)) / (|A| + |B|), el promedio de todas las distancias entre elementos de los dos grupos.

Buscar el mínimo cuesta $O(n^2)$ y actualizar una fila $O(n)$, así que el algoritmo completo es $O(n^3)$ en tiempo y $O(n^2)$ en espacio; con n = 20 tarda menos de un milisegundo. Para mostrar las actualizaciones tomamos cinco artículos sobre IA generativa y evaluación (11, 15, 16, 17 y 19), cuya matriz de distancias está en la Tabla 10.

Tabla 10

Distancias TF-IDF (1 − coseno) entre cinco artículos

	A11	A15	A16	A17	A19
A11	0,0000	0,9400	0,8975	0,9146	0,9612
A15	0,9400	0,0000	0,9647	0,9756	0,9769
A16	0,8975	0,9647	0,0000	0,9709	0,9425
A17	0,9146	0,9756	0,9709	0,0000	0,9743
A19	0,9612	0,9769	0,9425	0,9743	0,0000

Los tres criterios empiezan igual, uniendo los artículos 11 y 16 (d = 0,8975), el par más cercano, en un grupo G1. Para actualizar la distancia de G1 al artículo 17, single linkage toma mín(0,9146; 0,9709) = 0,9146, complete linkage toma máx(0,9146; 0,9709) = 0,9709 y average linkage toma (0,9146 + 0,9709)/2 = 0,9428. Con esos valores, single y average linkage unen después el artículo 17 al grupo, mientras que complete linkage prefiere el 19, porque máx(0,9612; 0,9425) = 0,9612 es menor que 0,9709. La Tabla 11 muestra la secuencia completa.

Tabla 11

Fusiones del subconjunto con cada criterio

Paso	Single linkage	Complete linkage	Average linkage
1	A11 + A16 (0,8975)	A11 + A16 (0,8975)	A11 + A16 (0,8975)
2	A17 + G1 (0,9146)	A19 + G1 (0,9612)	A17 + G1 (0,9428)
3	A15 + G2 (0,9400)	A17 + G2 (0,9743)	A19 + G2 (0,9593)
4	A19 + G3 (0,9425)	A15 + G3 (0,9769)	A15 + G3 (0,9643)

Nota. G1 es el grupo creado en el paso 1, G2 el del paso 2, y así sucesivamente; entre paréntesis, la distancia de la fusión.

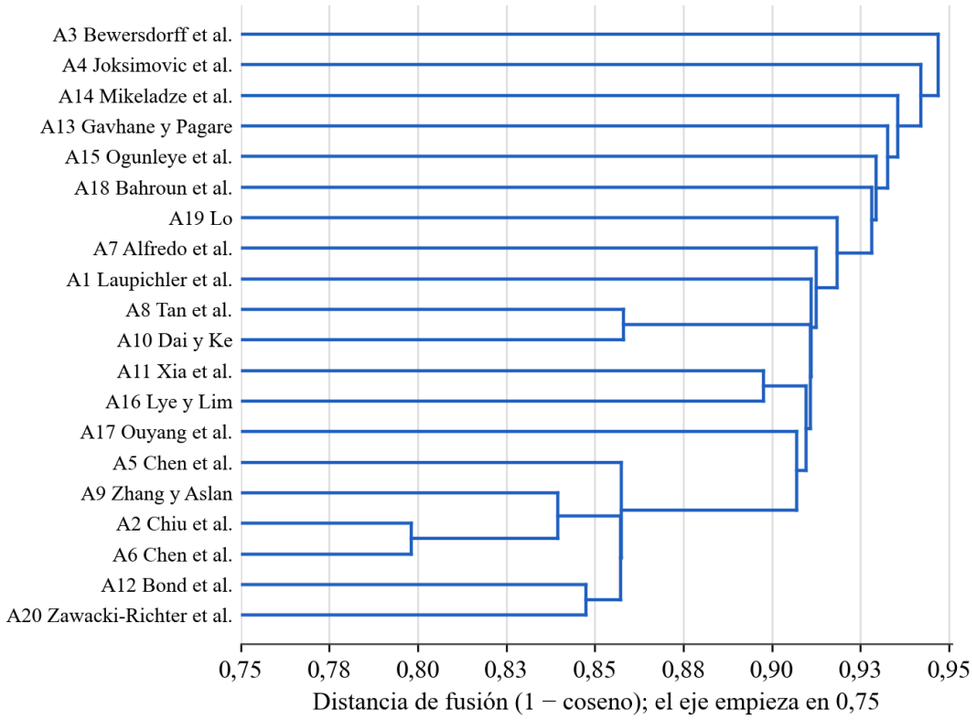
El ejemplo muestra el comportamiento característico de cada criterio: single linkage encadena, porque cada artículo se une al grupo por su vecino más cercano y las fusiones ocurren a distancias bajas; complete linkage solo une grupos cuando todos sus elementos están cerca, lo que eleva las distancias de fusión; y average linkage queda en un punto intermedio.

5.3 Dendrogramas

Las Figuras 4, 5 y 6 muestran los dendrogramas de los 20 abstracts con TF-IDF. El eje horizontal es la distancia a la que se fusionan dos grupos; como con TF-IDF todas las fusiones ocurren entre 0,79 y 0,98, el eje empieza en 0,75 para que se distinga la estructura. La aplicación dibuja los mismos dendrogramas en SVG, con los grupos coloreados según el corte que da la mejor silueta.

Figura 4

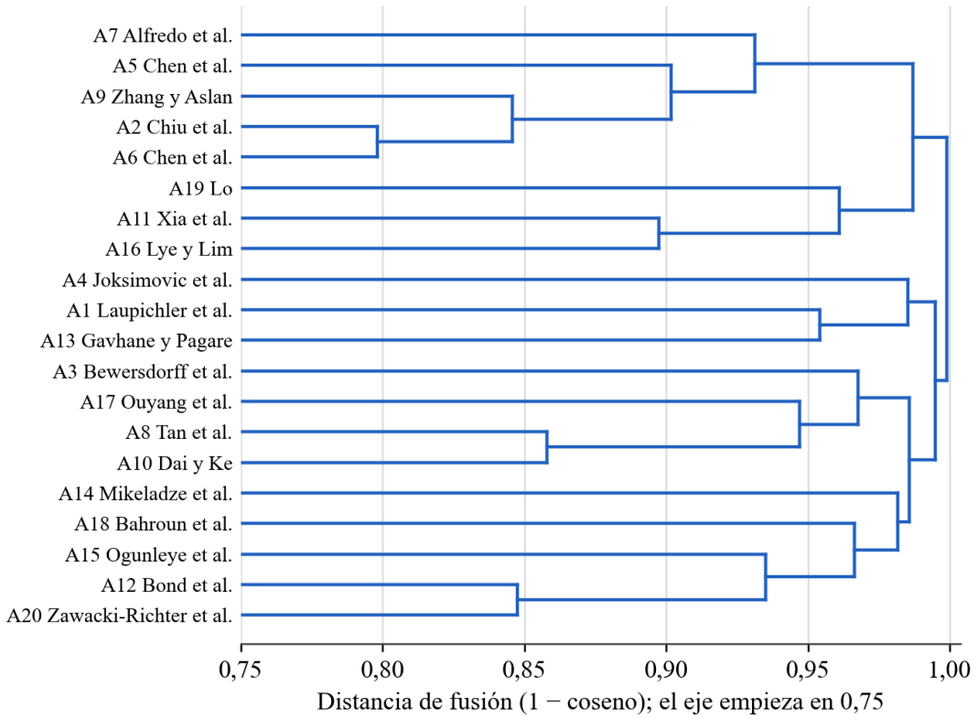
Dendrograma de los 20 abstracts con TF-IDF y single linkage



Nota. Distancia = 1 − similitud del coseno entre vectores TF-IDF. El eje horizontal empieza en 0,75. Elaboración propia.

Figura 5

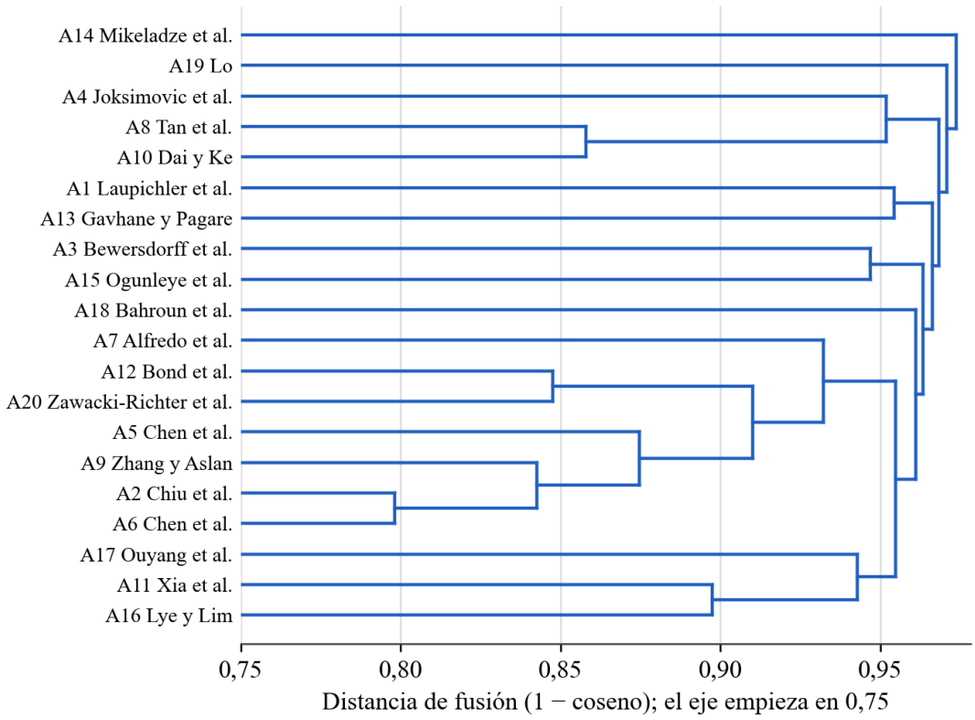
Dendrograma de los 20 abstracts con TF-IDF y complete linkage



Nota. Distancia = 1 − similitud del coseno entre vectores TF-IDF. El eje horizontal empieza en 0,75. Elaboración propia.

Figura 6

Dendrograma de los 20 abstracts con TF-IDF y average linkage



Nota. Distancia = 1 − similitud del coseno entre vectores TF-IDF. El eje horizontal empieza en 0,75. Elaboración propia.

En los tres árboles aparecen los mismos núcleos: los artículos 2 y 6, dos revisiones generales de la investigación en IA en la educación, se unen primero (d = 0,80) y se les suma enseguida el 9 y, según el criterio, el 5, de la misma línea; el 12 y el 20, revisiones sobre IA en la educación superior, forman otro par (0,85); el 8 y el 10, sobre técnicas de IA para el aprendizaje colaborativo y basado en simulación, un tercero (0,86); y el 11 y el 16, nuestro caso de estudio, un cuarto (0,90). Las diferencias están en cómo se unen esos núcleos: single linkage produce un árbol encadenado, en el que los artículos se agregan uno a uno a un gran grupo, y al cortarlo en dos grupos solo separa el artículo 3; complete linkage forma grupos más compactos y equilibrados (al cortarlo en dos quedan 8 y 12 artículos); y average linkage queda de nuevo entre los dos.

5.4 Evaluación interna: ¿qué criterio agrupa mejor?

Para decidir qué criterio produce agrupamientos más coherentes usamos dos métricas internas, que solo dependen de las distancias:

- **Coeficiente de correlación cofenética** (CCC; Sokal y Rohlf, 1962): la correlación de Pearson entre la distancia original de cada par de artículos y su distancia cofenética, que es la altura de la fusión en la que el par queda en el mismo grupo. Mide qué tan fielmente el dendrograma conserva las distancias; un CCC cercano a 1 indica un árbol fiel.
- **Coeficiente de silueta** (Rousseeuw, 1987): para cada artículo, s(i) = (b − a)/máx(a, b), donde a es su distancia media a los artículos de su grupo y b la menor distancia media a otro grupo. Se calcula al cortar el árbol en k grupos (k = 2 a 8) y va de −1 a 1; valores altos indican grupos compactos y separados.

Tabla 12

Correlación cofenética y mejor silueta de cada criterio según la representación

Representación	Criterio	CCC	Mejor k	Silueta con ese k
TF-IDF	Single linkage	0,738	2	0,014
TF-IDF	Complete linkage	0,688	8	0,022
TF-IDF	Average linkage	0,814	8	0,027
MiniLM	Single linkage	0,798	2	0,244
MiniLM	Complete linkage	0,527	4	0,119
MiniLM	Average linkage	0,801	2	0,208
GloVe	Single linkage	0,802	2	0,339
GloVe	Complete linkage	0,745	2	0,320
GloVe	Average linkage	0,826	2	0,339

Nota. Mejor k: número de grupos, entre 2 y 8, con la silueta más alta.

Average linkage obtiene el mayor CCC con las tres representaciones (0,814 con TF-IDF, 0,801 con MiniLM y 0,826 con GloVe), es decir, es el criterio cuyo dendrograma conserva mejor las distancias reales entre los abstracts. Es lo que cabe esperar: al promediar todas las distancias entre dos grupos, no se deja dominar por un único par muy cercano, como single linkage, que tiende a encadenar, ni por un único par muy lejano, como complete linkage, que con TF-IDF tiene el menor CCC (0,688). Por eso concluimos que average linkage produce los agrupamientos más coherentes para este corpus.

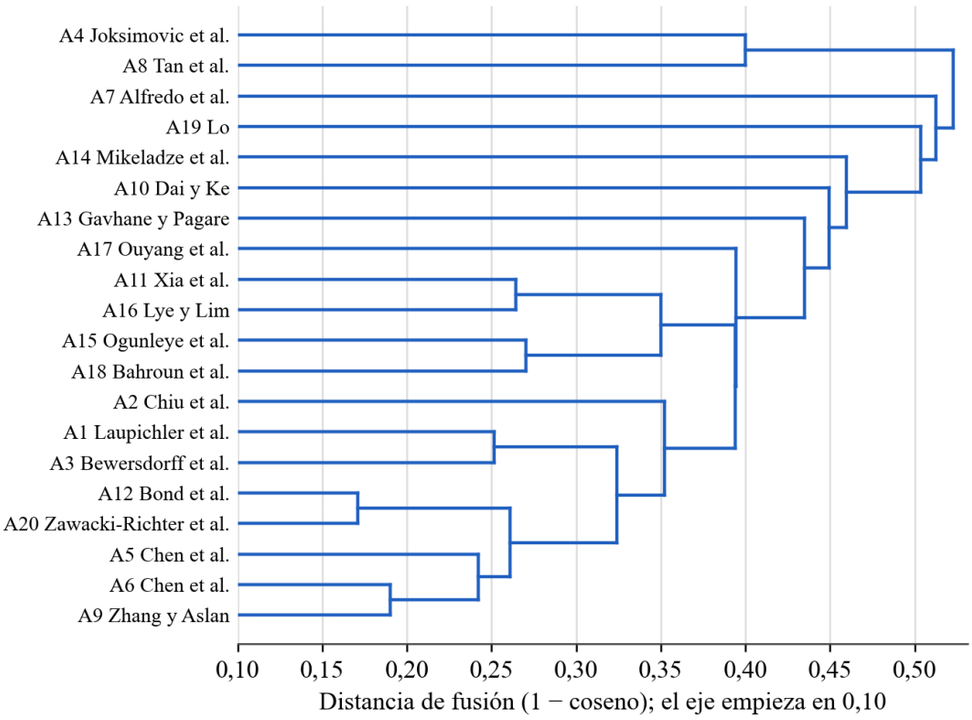
La silueta añade dos matices. El primero es que la representación pesa más que el criterio: con TF-IDF la silueta es muy baja para los tres criterios (menor que 0,03), porque

los abstracts comparten pocos términos informativos, todas las distancias están entre 0,80 y 1 y no hay grupos claramente separados; con los embeddings sube hasta 0,24 (MiniLM) y 0,34 (GloVe), porque estas representaciones sí capturan la cercanía temática. El segundo es que la silueta más alta no siempre corresponde a la partición más útil: con MiniLM, single y average linkage logran su mejor silueta con k = 2 separando del resto uno o dos artículos atípicos (el 4, sobre resolución de problemas complejos, y el 8, sobre aprendizaje colaborativo), mientras que complete linkage con k = 2 produce la partición temática más clara: de un lado los artículos sobre IA generativa, ChatGPT, evaluación y docencia (2, 11, 13 a 19) y del otro las revisiones generales de la investigación en IA en la educación (1, 3 a 10, 12 y 20).

La Figura 7 muestra el dendrograma de average linkage con MiniLM. Además de los núcleos que ya encontraba TF-IDF (12 y 20; 6, 9 y 5; 11 y 16), MiniLM une pares que TF-IDF no relacionaba y que tienen sentido temático: el 1 y el 3 (alfabetización en IA y concepciones erróneas sobre la IA) y el 15 y el 18 (dos revisiones sobre IA generativa en la enseñanza), aunque usen vocabulario distinto.

Figura 7

Dendrograma de los 20 abstracts con embeddings de MiniLM y average linkage



Nota. Distancia = 1 − similitud del coseno entre los embeddings de all-MiniLM-L6-v2. Elaboración propia.

6. Requerimiento 3: búsqueda de patrones

La aplicación busca todas las ocurrencias exactas de un patrón en los 20 abstracts con tres algoritmos implementados sin funciones de búsqueda del lenguaje; la opción de ignorar mayúsculas solo convierte el texto y el patrón a minúsculas carácter por carácter antes de buscar. Las pruebas unitarias comparan los tres algoritmos con una búsqueda por fuerza bruta en 300 textos aleatorios, y la aplicación verifica en cada consulta que los tres encuentren las mismas posiciones.

6.1 Caso de estudio

Tomamos el patrón P = «analysis» (m = 8) y la frase T = «This study begins with a content analysis that explores GAI's transformative impact» del abstract del artículo 18 (Bahroun et al., 2023), en minúsculas (n = 83 caracteres). El patrón aparece una vez, en la posición 33 (contando desde 0). La frase tiene además varios comienzos parciales del patrón,

como la «a» de «a content», de «that» o de «impact» y el «an» de «transformative», que ponen a prueba las estructuras auxiliares de cada algoritmo.

6.2 Knuth-Morris-Pratt

KMP (Knuth et al., 1977) evita volver a leer caracteres del texto. Su preprocesamiento calcula la función de prefijos π del patrón: π[q] es la longitud del prefijo propio más largo de P[0..q] que también es sufijo de P[0..q]. Si después de q coincidencias hay una discrepancia, el patrón se desplaza para alinear su prefijo de longitud π[q − 1] con lo ya leído, sin retroceder en el texto.

Tabla 13

Función de prefijos de «analysis»

q	0	1	2	3	4	5	6	7
P[q]	a	n	a	l	y	s	i	s
π[q]	0	0	1	0	0	0	0	0

Para «analysis» solo π[2] = 1: el prefijo «a» reaparece como sufijo de «ana»; el resto vale 0 porque ninguna otra parte del patrón repite su comienzo. La Tabla 14 muestra la búsqueda. Cada coincidencia aumenta q y cada discrepancia con q > 0 hace q = π[q − 1]. En i = 66, por ejemplo, el texto tiene «ans» y el patrón espera «ana»: con q = 2 falla la «s» y q pasa a π[1] = 0, sin mover hacia atrás el índice del texto. La ocurrencia se completa en i = 40, es decir, en la posición 40 − 8 + 1 = 33.

Tabla 14

Pasos de KMP sobre la frase del caso de estudio

i	T[i]	q antes	Acción	q después
23	a	0	coincide con P[0]	1
24	␣	1	falla frente a P[1] = «n»; q = π[0]	0
33	a	0	coincide con P[0]	1
34	n	1	coincide con P[1]	2
35	a	2	coincide con P[2]	3
36	l	3	coincide con P[3]	4
37	y	4	coincide con P[4]	5
38	s	5	coincide con P[5]	6
39	i	6	coincide con P[6]	7
40	s	7	coincide con P[7]	8
		8	ocurrencia en la posición 33; q = π[7]	0
44	a	0	coincide con P[0]	1
45	t	1	falla frente a P[1] = «n»; q = π[0]	0
57	a	0	coincide con P[0]	1
58	i	1	falla frente a P[1] = «n»; q = π[0]	0
64	a	0	coincide con P[0]	1
65	n	1	coincide con P[1]	2
66	s	2	falla frente a P[2] = «a»; q = π[1]	0
71	a	0	coincide con P[0]	1
72	t	1	falla frente a P[1] = «n»; q = π[0]	0
80	a	0	coincide con P[0]	1
81	c	1	falla frente a P[1] = «n»; q = π[0]	0

Nota. Solo se muestran los pasos con alguna coincidencia; en las demás posiciones q = 0, la comparación con P[0] falla y se avanza al siguiente carácter. ␣ representa un espacio.

En total se hicieron 89 comparaciones de caracteres para 83 caracteres de texto, dentro de la cota de 2n del algoritmo. El preprocesamiento es Θ(m) y la búsqueda Θ(n): el índice del texto solo avanza y el número de veces que q retrocede está acotado por el número de veces que avanzó (Cormen et al., 2022).

6.3 Boyer-Moore

Boyer-Moore (Boyer y Moore, 1977) compara el patrón de derecha a izquierda y, ante una discrepancia, lo desplaza lo máximo que permitan dos heurísticas, cuyas tablas se calculan en el preprocesamiento (Tablas 15 y 16):

- **Carácter malo**: si el carácter c del texto no coincide con P[j], el patrón se mueve j − última(c) posiciones para alinear la última aparición de c en el patrón; si c no aparece, última(c) = −1 y el patrón salta por completo.
- **Sufijo bueno**: si ya coincidió el sufijo P[j+1..m−1], el patrón se mueve hasta la siguiente aparición de ese sufijo en el patrón o del prefijo más largo que sea sufijo de esa parte. desplazamiento[j + 1] guarda ese movimiento y desplazamiento[0] el que se usa después de una ocurrencia completa.

Tabla 15

Tabla de carácter malo de «analysis»

c	a	i	l	n	s	y	otro
última(c)	2	6	3	1	7	4	-1

Tabla 16

Tabla de sufijo bueno de «analysis»

j	0	1	2	3	4	5	6	7	8
desplazamiento[j]	8	8	8	8	8	8	8	2	1

Nota. desplazamiento[j] se usa cuando ya coincidió P[j..7] y falló P[j − 1].

La tabla de sufijo bueno de «analysis» vale casi siempre 8 porque el patrón no tiene un borde, es decir, un prefijo que también sea sufijo; solo cuando ya coincidió la «s» final (desplazamiento[7] = 2) se puede alinear la otra «s» del patrón, y desplazamiento[8] = 1 es el caso en que no coincidió nada. La Tabla 17 muestra las alineaciones.

Tabla 17

Alineaciones de Boyer-Moore sobre la frase del caso de estudio

s	Falla	Coincidencias	Carácter malo	Sufijo bueno	Desplazamiento
0	«u» en j = 7	0	8	1	8
8	«n» en j = 7	0	6	1	6
14	«h» en j = 7	0	8	1	8
22	«e» en j = 7	0	8	1	8
30	«y» en j = 7	0	3	1	3
33	ocurrencia	8	–	–	8
41	«x» en j = 7	0	8	1	8
49	«g» en j = 7	0	8	1	8
57	«a» en j = 7	0	5	1	5
62	«r» en j = 7	0	8	1	8
70	«i» en j = 7	0	1	1	1
71	«m» en j = 7	0	8	1	8

Nota. s es la posición del texto donde empieza el patrón; el desplazamiento es el máximo de las dos heurísticas.

En casi todas las alineaciones el último carácter del patrón ya falla frente a un carácter que no aparece en él («u», «h», «e», «x»…), así que el patrón salta 8 posiciones de una vez. Solo se examinaron 12 alineaciones y se hicieron 19 comparaciones para 83 caracteres, menos de una por carácter: es el comportamiento sublineal de Boyer-Moore en texto natural, cuyo mejor caso es O(n/m). El peor caso de esta versión es O(n·m), por ejemplo al buscar «aaa» en «aaaa…», algo que no ocurre con texto en inglés.

6.4 Rabin-Karp

Rabin-Karp (Karp y Rabin, 1987) interpreta cada ventana de m caracteres como un número en base d = 256 y lo reduce módulo un primo q. El hash del patrón se calcula con el método de Horner y el de cada ventana se obtiene del anterior en O(1), quitando el carácter que sale y agregando el que entra:

$$h(s+1) = (d \cdot (h(s) - T[s] \cdot d^{m-1}) + T[s+m]) \bmod q$$

Si el hash de la ventana coincide con el del patrón se verifican los caracteres, porque dos cadenas distintas pueden tener el mismo hash. Para que esas colisiones se vean en el ejemplo usamos q = 13; la aplicación usa q = 1.000.000.007. Con q = 13, $d^{m-1} \bmod q = 256^7$

mod 13 = 9, y el hash del patrón se calcula letra por letra: «a» (97) da 97 mod 13 = 6; «n» (110) da (256 · 6 + 110) mod 13 = 8; «a» da 0; «l» da 4; «y» da 1; «s» da 7; «i» da 12; y la última «s» da h(P) = 2. La Tabla 18 muestra las ventanas alrededor de la ocurrencia y las que tienen el mismo hash que el patrón.

Tabla 18

Ventanas de Rabin-Karp con q = 13

s	Ventana	h(ventana)	Resultado
30	«nt␣analy»	7	
31	«t␣analys»	4	
32	«␣analysi»	1	
33	«analysis»	2	ocurrencia
34	«nalysis␣»	6	
35	«alysis␣t»	9	
36	«lysis␣th»	11	
37	«ysis␣tha»	2	colisión
38	«sis␣that»	5	
…	…	…	
62	«transfor»	8	
63	«ransform»	2	colisión
64	«ansforma»	7	
65	«nsformat»	5	
66	«sformati»	2	colisión

Nota. Se verifican los caracteres solo cuando h(ventana) = h(P) = 2. ␣ representa un espacio.

Con q = 13 hubo 3 colisiones en las 76 ventanas («ysis tha», «ransform» y «sformati»), que la verificación descartó con 11 comparaciones en total; con q = 1.000.000.007 no hubo ninguna y solo se compararon los 8 caracteres de la ocurrencia. El costo esperado es O(n + m), pero en el peor caso, si todas las ventanas colisionan, es O(n·m).

6.5 Análisis crítico: teoría frente a ejecución

La Tabla 19 resume la complejidad teórica de los tres algoritmos. Los tres tienen preprocesamiento lineal en m (más el alfabeto en Boyer-Moore) y, en el caso típico, búsqueda lineal en n; las diferencias están en el peor caso y en el costo de cada paso.

Tabla 19

Complejidad de los algoritmos de búsqueda

Algoritmo	Preprocesamiento	Búsqueda, peor caso	Búsqueda, caso típico	Espacio adicional
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$	$\Theta(n)$	$\Theta(m)$
Boyer-Moore	$\Theta(m + \sigma)$	$O(n \cdot m)$	entre $O(n/m)$ y $O(n)$	$\Theta(m + \sigma)$
Rabin-Karp	$\Theta(m)$	$O(n \cdot m)$	$O(n + m)$	$O(1)$

Nota. n = longitud del texto, m = longitud del patrón y σ = tamaño del alfabeto (256 en la tabla de carácter malo).

Para contrastar la teoría medimos los tres algoritmos sobre los 20 abstracts unidos (n = 28.270 caracteres) con patrones de 2 a 36 caracteres tomados del corpus (Tabla 20) y con el patrón «learning» sobre ese texto repetido 1, 10 y 100 veces (Figura 8). Cada tiempo es la mediana de 15 ejecuciones tras 5 de calentamiento.

Tabla 20

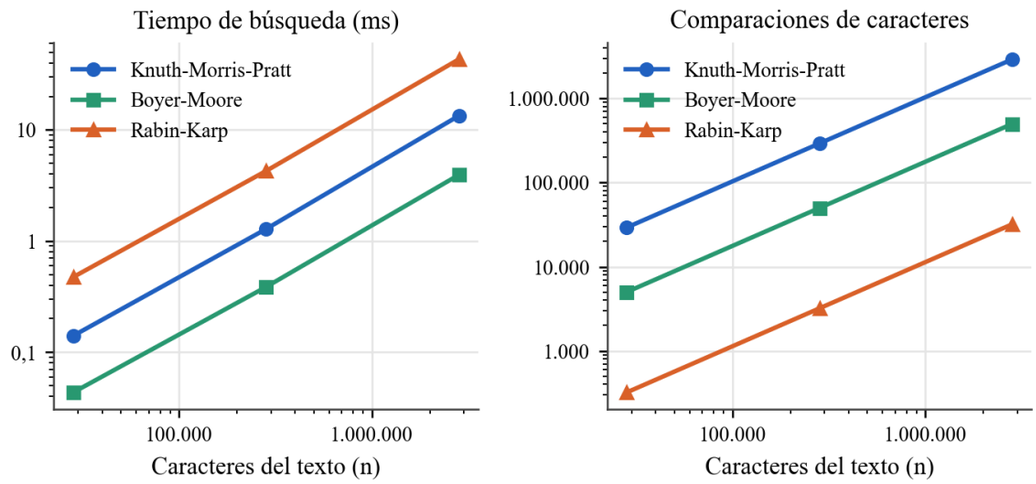
Comparaciones y tiempos según la longitud del patrón (n = 28.270 caracteres)

Patrón	m	Ocurr.	Comparaciones por carácter (KMP / BM / RK)	Búsqueda en ms (KMP / BM / RK)	Preproceso en µs (KMP / BM / RK)
«ai»	2	170	1,07 / 0,56 / 0,012	0,98 / 0,81 / 1,12	8,0 / 22,2 / 4,0
«data»	4	8	1,04 / 0,29 / 0,001	0,74 / 0,50 / 1,34	7,8 / 20,1 / 4,1
«learning»	8	40	1,03 / 0,18 / 0,011	0,68 / 0,34 / 1,40	12,2 / 34,9 / 4,9
«assessment»	10	37	1,07 / 0,15 / 0,013	0,37 / 0,36 / 1,34	9,1 / 19,1 / 4,3
«higher education»	16	14	1,03 / 0,15 / 0,008	0,15 / 0,30 / 3,78	1,1 / 18,5 / 4,1
«artificial intelligence»	23	22	1,07 / 0,09 / 0,018	0,16 / 0,21 / 0,49	0,6 / 7,8 / 0,5
«generative artificial intelligence»	34	5	1,02 / 0,07 / 0,006	0,13 / 0,18 / 0,47	0,4 / 3,0 / 0,5
«artificial intelligence in education»	36	6	1,07 / 0,09 / 0,008	0,20 / 0,21 / 0,48	0,6 / 6,6 / 0,6

Nota. KMP = Knuth-Morris-Pratt, BM = Boyer-Moore y RK = Rabin-Karp. En Rabin-Karp las comparaciones solo ocurren al verificar las ventanas cuyo hash coincide; no hubo colisiones con q = 1.000.000.007.

Figura 8

Tiempo de búsqueda y comparaciones según el tamaño del texto



Nota. Patrón «learning» (m = 8) sobre los 20 abstracts unidos y repetidos 1, 10 y 100 veces. Ejes en escala logarítmica. Elaboración propia.

Las mediciones confirman la teoría en lo que se puede contar exactamente. KMP hace siempre algo más de una comparación por carácter (entre 1,02 y 1,07), sin importar el patrón. Boyer-Moore hace cada vez menos a medida que el patrón crece, de 0,56 por carácter con «ai» a 0,07 con «generative artificial intelligence», porque cada salto puede avanzar hasta m posiciones. Rabin-Karp casi no compara caracteres, porque solo verifica las ventanas cuyo hash coincide, pero calcula un hash para cada una de las $n - m + 1$ ventanas. Al multiplicar el texto por 10 y por 100, los tiempos y las comparaciones de los tres crecen en la misma proporción (Figura 8), como corresponde a una búsqueda lineal.

El tiempo real ordena los algoritmos de forma distinta a lo que sugieren las comparaciones. Con el texto de 2,8 millones de caracteres, Boyer-Moore tardó 3,9 ms, KMP 13,4 ms y Rabin-Karp 43,1 ms, aunque Rabin-Karp hizo 16 veces menos comparaciones que Boyer-Moore: cada paso del hash rodante cuesta varias multiplicaciones y operaciones módulo, mucho más que comparar dos caracteres. El preprocesamiento es despreciable frente a la búsqueda (como máximo 35 µs, frente a décimas o decenas de milisegundos de búsqueda); el de Boyer-Moore es el más caro de los tres porque, además de la tabla de sufijo bueno, inicializa la tabla de 256 caracteres, un costo que se recupera con creces en la búsqueda. La conclusión práctica es que para buscar palabras en texto natural Boyer-Moore

es el más eficiente, KMP el más predecible (nunca más de 2n comparaciones, sin importar el texto) y Rabin-Karp tiene sentido cuando se buscan muchos patrones a la vez, porque el hash de cada ventana se puede comparar con los de todos los patrones en la misma pasada. Para comprobarlo agregamos a la aplicación Rabin-Karp multipatrón, que agrupa los patrones por longitud, recorre el texto una vez por cada longitud y busca el hash de cada ventana en una tabla hash de hash → patrones. Con diez patrones de ocho letras sobre el texto de 282.700 caracteres hizo una sola pasada y tardó 3,99 ms, frente a 12,88 ms de KMP, 5,92 ms de Boyer-Moore y 31,59 ms de Rabin-Karp ejecutados una vez por patrón; con diez patrones de cinco longitudes distintas necesitó cinco pasadas y tardó 19,48 ms, menos que Rabin-Karp por patrón (31,72 ms) pero más que KMP (13,39 ms) y Boyer-Moore (7,02 ms) (mediana de 15 ejecuciones; resultados/busqueda_multiple.csv).

6.6 Demostración paso a paso en la aplicación

La pestaña de búsqueda permite repetir esta demostración con cualquier ocurrencia del patrón (Figura 9). Al elegir una ocurrencia, la aplicación toma la oración del abstract que la contiene y muestra, para el algoritmo elegido, sus estructuras auxiliares (la función de prefijos, las tablas de carácter malo y de sufijo bueno, o la base, el módulo, d^(m−1) mod q y h(P)) y cada alineación del patrón sobre el texto: qué caracteres se comparan, cuáles coinciden, dónde falla y cuánto se desplaza el patrón según la regla que corresponda. En Rabin-Karp se puede elegir q = 13, para ver colisiones, o q = 1.000.000.007, el módulo de la búsqueda real.

La clase PasoAPaso recorre el texto con los mismos ciclos que las implementaciones y usa sus mismas funciones (funcionPrefijos, tablaCaracterMalo, tablaSufijoBueno, hash y rodar); una prueba unitaria verifica en 300 textos aleatorios que encuentra las mismas posiciones y hace el mismo número de comparaciones que cada algoritmo, así que lo que se ve en pantalla es el funcionamiento real y no una animación aparte. Sobre la oración completa del caso de estudio (171 caracteres), KMP hace 184 comparaciones en 162 alineaciones, Boyer-Moore 35 comparaciones en 28 alineaciones y Rabin-Karp, con q = 13, solo 17

comparaciones, aunque recorre 164 ventanas y tiene 7 colisiones. El enlace #pasos=analysis,18,426 abre directamente este caso.

Figura 9

Interfaz web: paso a paso de KMP en el caso de estudio



Nota. Captura de la aplicación en ejecución. Verde: caracteres que coinciden; rojo: el que falla; azul: los que KMP ya sabe que coinciden y no vuelve a comparar. Elaboración propia.

7. Ranking de términos con ordenamientos no comparativos

Este capítulo integra al proyecto final el ranking de términos del seguimiento, calculado ahora sobre el mismo corpus y el mismo preprocesamiento del resto de la aplicación, con las correcciones que pidió el docente: el programa de verificación en Python está en el repositorio (sección 7.7 y Anexo C); cada medición se repitió en 20 o 30 ejecuciones independientes y se reporta con su mediana, su rango intercuartílico y tablas numéricas (secciones 7.4 a 7.6); y el efecto de la memoria caché se estudia con un experimento controlado (sección 7.6). El ranking tiene además su propia pestaña en la interfaz y su consulta en la API (Tabla 2).

7.1 Lista de frecuencias

Los términos se obtienen con el preprocesamiento de la sección 3.2 aplicado a los 20 abstracts: normalización, minúsculas, tokenización por caracteres que no son letras y eliminación de stopwords y de tokens de una letra. No se aplica stemming, para que el ranking muestre palabras completas, aunque eso signifique que «study» y «studies» se cuenten por

separado. El conteo usa un mapa término → frecuencia que recorre los artículos del 1 al 20 y conserva el orden en que cada término aparece por primera vez, un detalle que explica cómo se resuelven los empates (sección 7.7). La Tabla 21 resume las cifras.

Tabla 21

Cifras del corpus para el ranking de términos

Indicador	Valor
Artículos	20
Palabras en los abstracts	4.007
Términos tras el preprocesamiento	2.407
Términos distintos (n)	950
Frecuencia máxima / mínima	86 / 1
Rango de claves k = máx − mín + 1	86
Términos que aparecen una sola vez	575 (60,5 %)

Nota. n es el número de elementos que se ordenan y k, el rango de valores que puede tomar la clave.

La distribución es muy desigual: más de la mitad de los términos aparece una sola vez, mientras que «ai» aparece 86 veces. Para el análisis que sigue, lo que más importa de estas cifras es que k = 86 es mucho menor que n = 950.

7.2 Fundamentación teórica

Counting Sort, Radix Sort y Bucket Sort pertenecen al grupo de los ordenamientos que no comparan elementos entre sí (GeeksforGeeks, s. f.): usan la clave, aquí la frecuencia, como índice de un arreglo. Eso es posible porque la clave es un entero dentro de un rango conocido, de 1 a 86 en este corpus, y es lo que les permite bajar de O(n log n) a O(n + k) u O(n) (Cormen et al., 2022). Un algoritmo que ordena comparando pares de elementos, como los métodos de burbuja, selección o MergeSort vistos en el curso (Cardona Torres, s. f.), se puede representar con un árbol de decisión binario cuyas hojas son las n! formas posibles de ordenar la entrada; su altura, que es el número de comparaciones en el peor caso, cumple:

$$h \geq \log_2(n!) = \Omega(n \log n)$$

Para n = 950 eso significa al menos 8.033 comparaciones en el peor caso. Los tres algoritmos de este capítulo no están atados a ese límite porque no toman decisiones de sí o no: calculan la posición de cada elemento a partir de su clave.

Counting Sort cuenta cuántos términos tienen cada frecuencia y con esos conteos calcula dónde empieza cada grupo en la salida. Hace tres recorridos: el primero incrementa conteo[f − mín] por cada término, con costo O(n); el segundo recorre los conteos de la frecuencia mayor a la menor y convierte cada uno en la posición inicial de su grupo, con costo O(k); y el tercero vuelve a pasar por la entrada y deja cada término en su posición, otra vez O(n). El espacio adicional es O(n + k) y el algoritmo es estable, porque el tercer recorrido respeta el orden de la entrada.

$$T(n, k) = O(n) + O(k) + O(n) = O(n + k)$$

Radix Sort ordena dígito por dígito, empezando por las unidades (variante LSD), con diez colas enlazadas, una por dígito, como en la explicación del curso (Algoritmo de ordenamiento RadixSort, s. f.). Cada pasada pone cada término en la cola de su dígito actual y después vacía las colas en orden, con un costo O(n + b) para la base b = 10. Si la frecuencia máxima tiene d dígitos:

$$T(n) = O(d \cdot (n + b)), \quad d = \lfloor \log_b f_máx \rfloor + 1$$

Con f_máx = 86 bastan d = 2 pasadas y, como ni d ni b dependen de n, el costo es O(n). El resultado es correcto porque cada pasada es estable: las colas son FIFO y dos términos con el mismo dígito salen en el orden en que llegaron.

Bucket Sort reparte los n términos en n cubetas con un índice que se calcula sin comparar, ordena cada cubeta por inserción y al final las concatena:

$$i = \lfloor (f - f_mín) \cdot n / k \rfloor, \quad T(n) = O(n) + \Sigma\, O(n_i^2)$$

Si las claves se reparten de manera uniforme, el valor esperado de n_i^2 es 2 − 1/n y el costo esperado es O(n) (Cormen et al., 2022); el peor caso, con todas las claves en la misma cubeta, es $O(n^2)$. En este corpus ese caso no puede ocurrir: como k ≤ n, los índices de dos

frecuencias distintas difieren en al menos $n/k \geq 1$, así que cada cubeta recibe un solo valor de frecuencia, la inserción no mueve ningún elemento y el costo es O(n) incluso en el peor caso. La Tabla 22 reúne las complejidades y la condición que cada algoritmo necesita para trabajar en tiempo lineal.

Tabla 22

Complejidad de los algoritmos de ordenamiento y condición para obtener O(n)

Algoritmo	Tiempo	Espacio adicional	Estable	Condición para O(n)	En este corpus
Counting Sort	O(n + k)	O(n + k)	Sí	k = O(n)	k = 86 ≤ n = 950
Radix Sort (LSD, base 10)	O(d · (n + b))	O(n + b)	Sí	d y b constantes	d = 2, b = 10
Bucket Sort (n cubetas)	O(n) esperado; $O(n^2)$ en el peor caso	O(n)	Sí	Claves uniformes, o k ≤ n	k ≤ n: una frecuencia por cubeta

Nota. n = número de términos, k = rango de frecuencias, d = dígitos de la frecuencia máxima, b = base.

7.3 Implementación

El paquete ranking (Tabla 1) reutiliza el lector del corpus y el preprocesador de los demás requerimientos, de modo que el ranking se calcula sobre exactamente los mismos abstracts. Los tres algoritmos implementan la interfaz AlgoritmoOrdenamiento, que recibe un arreglo de objetos Termino(palabra, frecuencia) y devuelve un arreglo nuevo, ordenado de mayor a menor, sin modificar el original; su código está en el Anexo B. Ningún algoritmo compara dos términos para decidir cuál va primero: la única comparación entre elementos está dentro de las cubetas de Bucket Sort y, como se vio en la sección 7.2, con estos datos nunca produce desplazamientos. Para obtener el orden descendente, Counting Sort asigna las posiciones desde la frecuencia más alta, Radix Sort vacía las colas de la 9 a la 0 y Bucket Sort concatena desde la última cubeta.

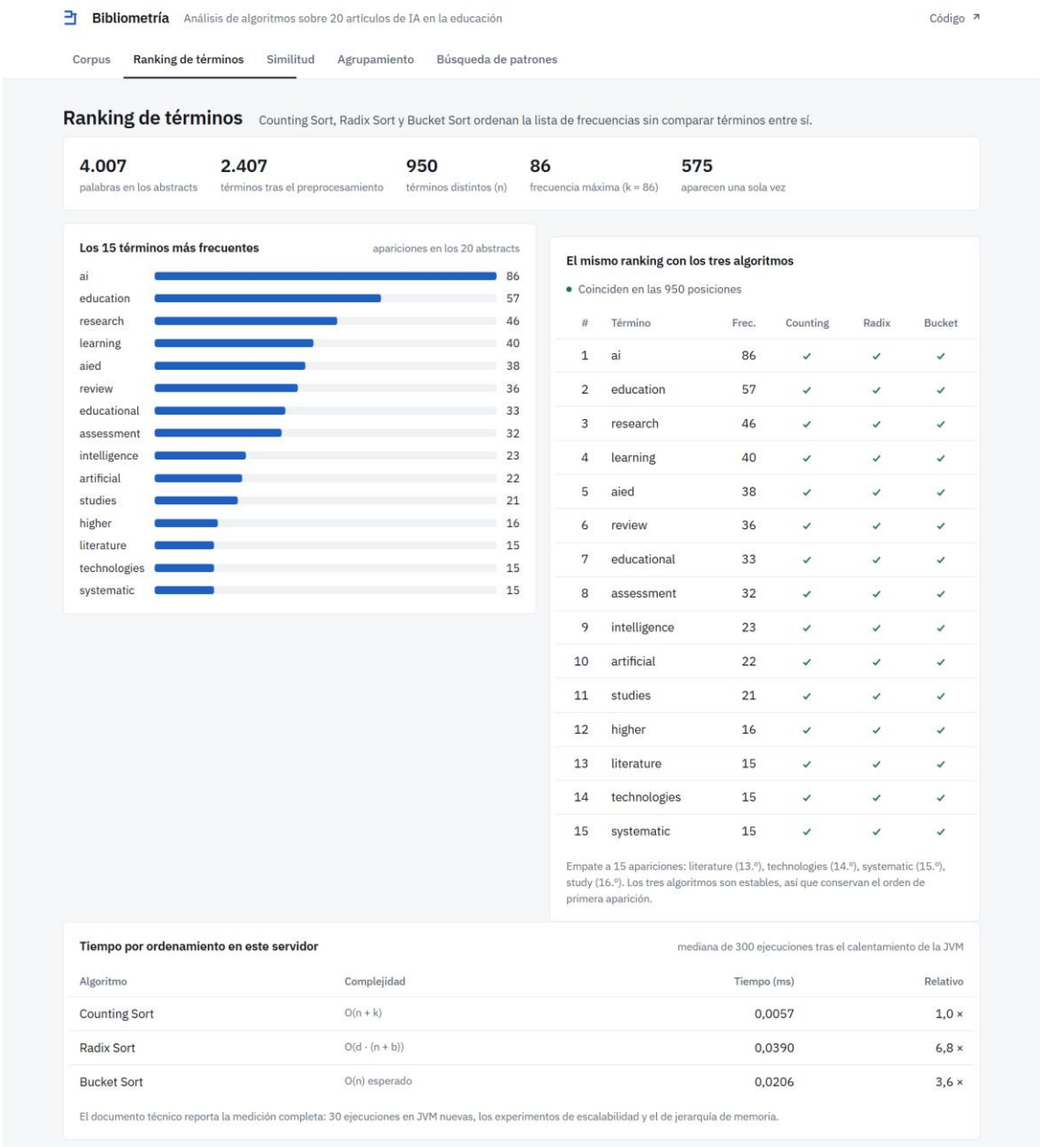
Los tres son estables y la lista de entrada sigue el orden de primera aparición, así que los términos con la misma frecuencia quedan en el mismo orden en los tres resultados. Las pruebas unitarias comparan cada algoritmo con el ordenamiento estable de Java (Arrays.sort) en 24 listas aleatorias de hasta 20.000 elementos, con rangos de 1 a 1.000.000, y en una lista

con distribución tipo Zipf. La clase RankingTerminos ejecuta la medición completa de las secciones 7.4 a 7.6 y guarda los resultados en la carpeta resultados/ranking del repositorio, y el script docs/generar_figuras_ranking.py dibuja con Matplotlib (Hunter, 2007) las figuras de este capítulo a partir de esos archivos.

En la interfaz, la pestaña Ranking de términos (Figura 10) muestra las cifras del corpus, los 15 términos más frecuentes y la comprobación, posición por posición, de que los tres algoritmos producen el mismo ranking. También mide en el servidor el tiempo de cada algoritmo, como mediana de 300 ordenamientos después de calentar la JVM; esos tiempos dependen del equipo que atiende la consulta, y la medición controlada es la de la sección 7.4.

Figura 10

Interfaz web: pestaña del ranking de términos



Nota. Captura de la aplicación en ejecución. Los tiempos de la última tabla se miden en el servidor en cada consulta; la medición completa está en la sección 7.4. Elaboración propia.

7.4 Medición de tiempos

Sobre la lista de 950 frecuencias, Counting Sort tardó 0,0052 ms por ordenamiento, Radix Sort 0,0308 ms y Bucket Sort 0,0173 ms (Tabla 23). Los tres quedan muy por debajo de un milisegundo, aunque Counting Sort resultó 5,9 veces más rápido que Radix Sort y 3,3 veces más rápido que Bucket Sort.

Cada ordenamiento se cronometró con System.nanoTime(). Como la JVM interpreta el código al principio y lo compila a código nativo (JIT) después de varias ejecuciones,

reportamos dos medidas: en frío, la primera ejecución de cada algoritmo al arrancar el programa, y en caliente, la mediana de 1.000 ejecuciones tomadas después de 5.000 rondas de calentamiento. Las ejecuciones se intercalan entre los tres algoritmos (Counting, Radix, Bucket, Counting...) para que una pausa del recolector de basura no perjudique solo a uno. El programa se corrió 30 veces, cada vez en una JVM nueva, y la tabla muestra la mediana de esas 30 ejecuciones con su rango intercuartílico (RIC, del primer al tercer cuartil), que indica cuánto varió cada medida de una ejecución a otra; los cuartiles se calculan con interpolación lineal (Hyndman y Fan, 1996). Los experimentos de las secciones 7.5 y 7.6 se repitieron 20 veces cada uno, también en JVM nuevas, y se reportan de la misma forma.

El equipo de medición es el de los demás capítulos (AMD Ryzen 7 5700U de 8 núcleos, Java 21 y Windows 11), conectado a la corriente. Según el sistema operativo, cada núcleo tiene 32 KiB de caché L1 de datos y 512 KiB de caché L2, y la caché L3 es de 8 MiB, repartida en dos bloques de 4 MiB, uno por cada grupo de cuatro núcleos.

Tabla 23

Tiempos de ordenamiento sobre la lista de frecuencias del corpus (n = 950, k = 86)

Algoritmo	En frío (ms)	En caliente (ms)	RIC en caliente (ms)	Mín. – máx. en caliente (ms)	Relativo a Counting Sort
Counting Sort	0,254	0,0052	0,0049 – 0,0056	0,0047 – 0,0071	1,0 ×
Radix Sort	1,641	0,0308	0,0295 – 0,0325	0,0276 – 0,0408	5,9 ×
Bucket Sort	0,878	0,0173	0,0165 – 0,0182	0,0155 – 0,0250	3,3 ×

Nota. Mediana de 30 ejecuciones independientes, cada una en una JVM nueva. En caliente: mediana de 1.000 ordenamientos tras 5.000 rondas de calentamiento. RIC: primer y tercer cuartil de las 30 ejecuciones.

Los tres hacen trabajo lineal, así que la diferencia está en las constantes. Counting Sort solo usa dos arreglos de enteros y recorre la entrada cuatro veces (para hallar el mínimo, el máximo, contar y colocar). Radix Sort hace dos pasadas y en cada una crea un nodo de cola por término, 1.900 objetos en total. Bucket Sort reserva un arreglo de 950 cubetas y crea una lista por cada valor de frecuencia presente, que en este corpus son 27. Crear objetos cuesta más que escribir en un arreglo y, como muestra la sección 7.6, esos objetos además empeoran el uso de la memoria caché.

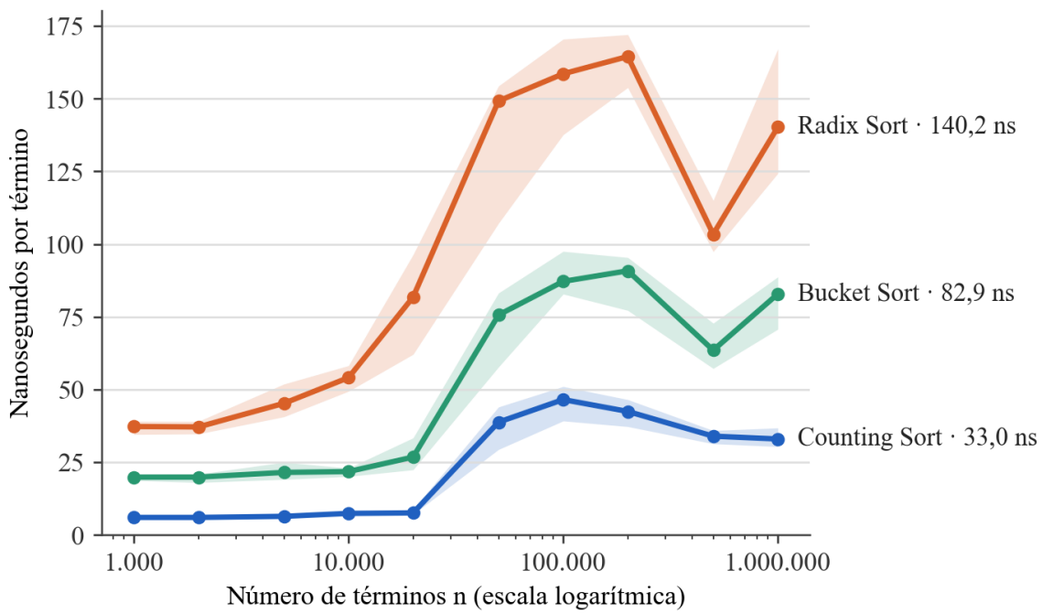
La primera ejecución fue entre 49 y 53 veces más lenta que las siguientes. Ese tiempo extra es del arranque de la JVM, que todavía interpreta el código y carga clases como ColaEnlazada y ArrayList, y no depende del algoritmo; por eso comparamos los tiempos en caliente. Hay que reconocer también que, con solo 950 términos, cualquiera de los tres sirve en la práctica: entre el más rápido y el más lento hay unos 26 microsegundos.

7.5 Comportamiento al aumentar n y k

Con 950 términos los tiempos son tan pequeños que no dejan ver la forma de la curva, así que hicimos un experimento adicional con listas sintéticas más grandes. En la primera parte crece n, de 1.000 a 1.000.000 en diez pasos, y las frecuencias se toman al azar de la distribución real del corpus. En la segunda, n queda fijo en 100.000 y lo que crece es el rango k, de 100 a 10.000.000. Cada punto es la mediana de 20 ejecuciones en JVM nuevas; la franja sombreada de las Figuras 11 y 12 es el RIC, y las Tablas 24 y 25 dan los valores numéricos.

Figura 11

Tiempo por término según el tamaño de la lista



Nota. Frecuencias muestreadas del corpus (k ≤ 86). Mediana de 20 ejecuciones; la franja sombreada es el rango intercuartílico. Eje horizontal en escala logarítmica. Elaboración propia.

Mientras los datos caben en la caché del procesador, el tiempo por término casi no cambia: de n = 1.000 a n = 20.000 pasa de 6,0 a 7,6 ns en Counting Sort y de 19,9 a 26,8 ns

en Bucket Sort, es decir, el tiempo total crece en la misma proporción que n, como corresponde a un algoritmo lineal. Entre 20.000 y 50.000 términos el costo por término se multiplica por 5,1 en Counting Sort, por 1,8 en Radix Sort y por 2,8 en Bucket Sort, y desde ahí se queda en un nivel más alto: en Counting Sort, entre 5,5 y 7,7 veces el costo con 1.000 términos (Tabla 24). Las fórmulas de la sección 7.2 no tienen ningún término que crezca más rápido que n, así que el número de operaciones por término es el mismo para cualquier n y el salto no puede venir de la complejidad del algoritmo; la sección 7.6 muestra con un experimento controlado que se debe a la jerarquía de memoria. La caída en n = 500.000, que aparece en los tres algoritmos, la atribuimos al mismo factor: la ubicación de los objetos en la memoria después de las recolecciones de basura.

Tabla 24

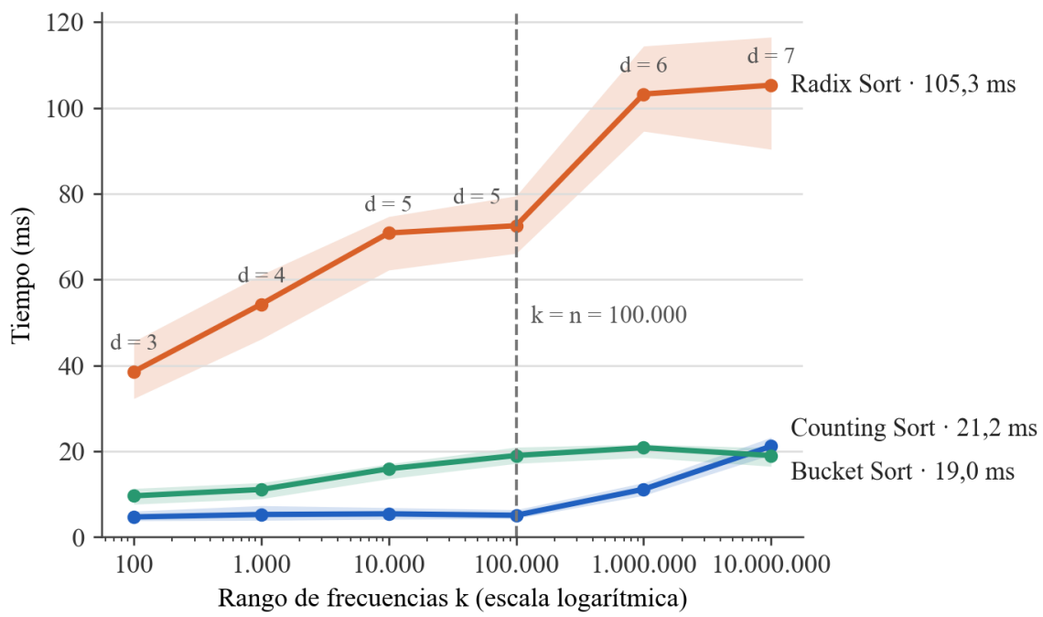
Nanosegundos por término según el tamaño de la lista (valores de la Figura 11)

n	Counting Sort	Radix Sort	Bucket Sort
1.000	6,0 (5,6–6,4)	37,3 (34,6–39,1)	19,9 (18,8–21,0)
2.000	6,0 (5,6–6,3)	37,2 (34,8–39,1)	19,9 (18,1–20,9)
5.000	6,4 (5,9–7,0)	45,3 (40,7–51,9)	21,5 (19,1–25,0)
10.000	7,4 (6,8–7,6)	54,1 (49,4–58,2)	21,8 (20,1–23,0)
20.000	7,6 (7,1–8,8)	81,8 (62,1–96,4)	26,8 (22,4–33,4)
50.000	38,8 (29,4–44,0)	149,2 (107,2–154,4)	75,7 (57,8–83,2)
100.000	46,6 (39,2–51,1)	158,6 (137,6–170,5)	87,2 (82,9–97,5)
200.000	42,5 (37,3–46,6)	164,5 (153,8–172,0)	90,8 (77,2–95,4)
500.000	34,0 (31,4–35,9)	103,4 (97,4–115,0)	63,5 (57,3–72,8)
1.000.000	33,0 (30,4–36,8)	140,2 (124,3–167,1)	82,9 (70,8–88,7)

Nota. Mediana de 20 ejecuciones y, entre paréntesis, su RIC.

Figura 12

Tiempo de ordenamiento según el rango de frecuencias, con n = 100.000



Nota. Mediana de 20 ejecuciones; la franja sombreada es el RIC. Elaboración propia.

La Figura 12 coincide con la teoría de la sección 7.2. Counting Sort casi no cambia mientras $k \leq n$ (entre 4,7 y 5,4 ms), pero con k = 1.000.000 tarda 11,1 ms y con k = 10.000.000, cien veces n, 21,2 ms, 4,5 veces más que con k = 100. Ahí pesan dos cosas: el término O(k), porque tiene que recorrer un arreglo de k conteos, y la memoria, porque ese arreglo ocupa 4 MB con k = 1.000.000 (lo mismo que la caché L3) y 40 MB con k = 10.000.000, de modo que cada incremento del histograma cae en una posición aleatoria fuera de la caché. Radix Sort depende del número de dígitos y no directamente de k: entre k = 10.000 y k = 99.998, ambos con d = 5, su tiempo no cambia aunque k se multiplica por diez, mientras que cada dígito adicional entre d = 3 y d = 6 le suma entre un 28 % y un 41 %. Entre d = 6 y d = 7 la diferencia medida (1 %) quedó por debajo de la esperada (17 %); a ese tamaño el costo de cada pasada lo dominan los accesos a los nodos de las colas enlazadas, que varían entre ejecuciones más que esa diferencia, aunque no logramos aislar la causa con este experimento. Bucket Sort siempre usa n cubetas: su tiempo sube de 9,6 a unos 21 ms a medida que aumentan las cubetas ocupadas, que pasan de 100 a unas 63.000, cada una con su propia lista, y deja de subir cuando k supera a n, porque a partir de ahí ya no aumentan las cubetas ocupadas (Tabla 25).

Tabla 25

Tiempo de ordenamiento en ms según el rango de frecuencias k, con n = 100.000 (valores de la Figura 12)

k	d	Counting Sort	Radix Sort	Bucket Sort
100	3	4,7 (3,8–6,0)	38,5 (32,3–45,6)	9,6 (7,7–11,3)
1.000	4	5,3 (3,9–7,3)	54,3 (46,1–61,1)	11,1 (8,9–12,7)
10.000	5	5,4 (4,2–6,8)	70,8 (62,2–74,7)	15,9 (13,6–17,0)
99.998	5	5,1 (4,3–6,4)	72,5 (66,1–79,5)	19,0 (17,2–20,9)
999.997	6	11,1 (9,7–12,7)	103,2 (94,5–114,4)	20,8 (18,5–21,6)
9.999.874	7	21,2 (19,2–23,2)	105,3 (90,3–116,5)	19,0 (16,5–20,8)

Nota. Mediana de 20 ejecuciones y, entre paréntesis, su RIC. k es la frecuencia máxima que resultó en la lista y d, su número de dígitos.

7.6 Efecto de la jerarquía de memoria

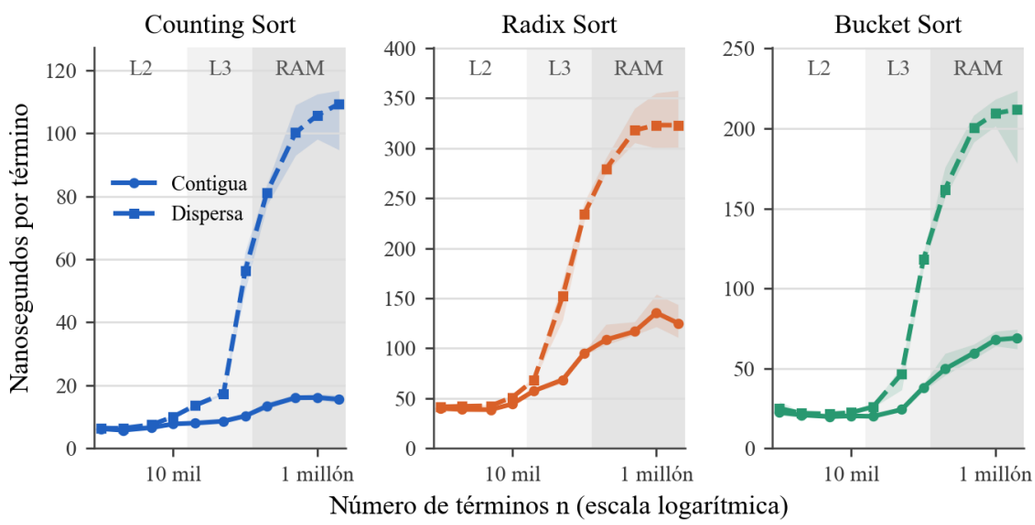
La notación O cuenta operaciones y supone que todas cuestan lo mismo, pero en un computador real el costo de leer un dato depende de dónde esté. Un dato que está en la caché L1 se lee en unos pocos ciclos, mientras que uno que hay que traer de la memoria principal tarda del orden de cien nanosegundos, entre 50 y 100 veces más (Hennessy y Patterson, 2019; Drepper, 2007). El procesador mueve la memoria en líneas de 64 bytes y, cuando detecta un recorrido secuencial, trae por adelantado las líneas siguientes (prefetching); si los accesos saltan a direcciones impredecibles, cada lectura puede ser un fallo de caché. En el equipo de medición, un solo hilo, como el de nuestras mediciones, cuenta con 512 KiB de L2 y con los 4 MiB de L3 de su grupo de núcleos.

Para saber desde qué tamaño dejan de caber los datos medimos cuánto ocupa un término en la memoria de la JVM: 28,45 bytes, que corresponden a los 24 bytes del objeto Termino (12 de encabezado, 4 de la referencia a la palabra y 4 de la frecuencia, más relleno hasta un múltiplo de 8) y a los 4 bytes de su referencia en el arreglo. Counting Sort escribe además una referencia en el arreglo de salida, así que recorre unos 32,5 bytes por término: los datos llenan la L2 alrededor de n = 16.000 y la L3 alrededor de n = 129.000. Por otra parte, Counting Sort lee la frecuencia de cada término cuatro veces (al buscar el mínimo y el máximo, al contar y al colocar) y escribe una referencia; esas cinco operaciones por término no dependen de n.

Para separar el efecto de la memoria del número de operaciones hicimos un experimento controlado (opción --memoria de RankingTerminos). Para cada n se arman dos listas con las mismas frecuencias en las mismas posiciones, así que cada algoritmo ejecuta exactamente las mismas instrucciones sobre ambas. Solo cambia dónde quedan los objetos en la memoria: en la lista contigua se crean en el orden del arreglo, de modo que recorrer el arreglo es recorrer la memoria en orden; en la lista dispersa se crean en un orden aleatorio, de modo que dos posiciones vecinas del arreglo apuntan a direcciones alejadas. Antes de medir se fuerza una recolección de basura completa, que compacta el heap sin alterar el orden relativo de los objetos y los pasa a la generación vieja, donde ya no se mueven. Cada punto es la mediana de 20 ejecuciones en JVM nuevas.

Figura 13

Tiempo por término con los objetos contiguos o dispersos en la memoria



Nota. Mismas frecuencias y mismas operaciones en las dos listas. Mediana de 20 ejecuciones; la franja sombreada es el RIC. Las zonas grises marcan desde qué n los datos (unos 32,5 bytes por término) dejan de caber en la caché L2 (512 KiB) y en la L3 (4 MiB). Cada panel tiene su propia escala vertical, porque la comparación es entre las dos listas de un mismo algoritmo. Elaboración propia.

La Figura 13 y la Tabla 26 confirman la explicación. Con n ≤ 5.000 (hasta 158 KiB de datos) las dos listas cuestan prácticamente lo mismo: la diferencia no pasa del 14 %, porque cualquier línea que se necesite ya está en la caché. Entre n = 10.000 y n = 50.000, cuando los datos ocupan más de la mitad de la L2 o ya no caben en ella y compiten con el resto de datos del programa, la lista dispersa cuesta entre 1,3 y 2,0 veces más en Counting

Sort. Con n = 100.000 los datos (3,1 MiB) en teoría todavía caben en la L3, pero la ocupan casi entera, y la razón salta a 5,5. Con un millón de términos (31 MiB, unas ocho veces la L3) Counting Sort tarda 106 ns por término con la lista dispersa y 16,1 ns con la contigua, 6,6 veces más, aunque ejecuta exactamente las mismas instrucciones. Con la lista contigua el costo también sube, pero mucho menos (de 6,2 a 15,6 ns), porque el prefetching anticipa las lecturas secuenciales y oculta buena parte de la espera. Radix Sort y Bucket Sort muestran lo mismo, con razones de hasta 2,7 y 3,4.

Tabla 26

Nanosegundos por término con los objetos contiguos y dispersos en la memoria (valores de la Figura 13)

n	Datos	Nivel	Counting Sort	Radix Sort	Bucket Sort
1.000	32 KiB	L2	6,2 / 6,4 (1,0×)	39,9 / 41,6 (1,0×)	22,4 / 25,0 (1,1×)
2.000	63 KiB	L2	5,8 / 6,4 (1,1×)	38,9 / 42,0 (1,1×)	20,9 / 22,1 (1,1×)
5.000	158 KiB	L2	6,6 / 7,5 (1,1×)	38,7 / 42,7 (1,1×)	19,9 / 21,5 (1,1×)
10.000	317 KiB	L2	7,8 / 10,0 (1,3×)	44,4 / 51,0 (1,2×)	20,3 / 22,4 (1,1×)
20.000	634 KiB	L3	8,0 / 13,6 (1,7×)	57,6 / 68,5 (1,2×)	20,0 / 26,0 (1,3×)
50.000	1,5 MiB	L3	8,6 / 17,4 (2,0×)	68,5 / 152 (2,2×)	24,3 / 46,7 (1,9×)
100.000	3,1 MiB	L3	10,3 / 56,4 (5,5×)	95,4 / 234 (2,5×)	38,1 / 118 (3,1×)
200.000	6,2 MiB	RAM	13,4 / 81,2 (6,1×)	109 / 279 (2,6×)	49,7 / 162 (3,3×)
500.000	15,5 MiB	RAM	16,1 / 100 (6,2×)	117 / 318 (2,7×)	59,6 / 200 (3,4×)
1.000.000	30,9 MiB	RAM	16,1 / 106 (6,6×)	135 / 323 (2,4×)	67,9 / 209 (3,1×)
2.000.000	61,9 MiB	RAM	15,6 / 109 (7,0×)	125 / 323 (2,6×)	69,0 / 212 (3,1×)

Nota. En cada celda: lista contigua / lista dispersa y, entre paréntesis, cuántas veces más tarda la dispersa. Mediana de 20 ejecuciones. Datos: bytes que recorre Counting Sort (32,5 por término). Nivel: caché en la que caben esos datos según su tamaño.

Esto explica la Figura 11. En ese experimento cada término tiene su propia palabra («t0», «t1»...), una cadena que la JVM crea justo antes del objeto Termino; por eso dos términos consecutivos quedan a unos 72 bytes de distancia y cada uno ocupa en la práctica una línea de caché completa. Con unos 72 bytes por término los datos dejan de caber en la L3 alrededor de n = 58.000, y la Figura 11 muestra el salto entre 20.000 y 50.000 términos, un poco antes, porque la L3 también guarda los arreglos y objetos que crea cada ordenamiento. Radix Sort y Bucket Sort sufren más que Counting Sort porque, además de leer los términos, crean un objeto por término (nodos de cola o elementos de las listas de cada

cubeta) en cada ordenamiento: sus datos ocupan más, salen antes de la caché y se recorren siguiendo referencias, en un orden que el procesador no puede anticipar.

En resumen, la complejidad O(n) describe bien el número de operaciones, que según la sección 7.2 es constante por término, pero el tiempo de cada operación depende del nivel de la jerarquía de memoria en el que estén los datos. Al comparar algoritmos de la misma complejidad, la localidad de los accesos, es decir, cuántos objetos crea cada uno y en qué orden los recorre, explica buena parte de las diferencias en las constantes: Counting Sort, que solo recorre arreglos en orden, es el que menos se ve afectado al crecer n.

7.7 Validación del ranking

Los tres algoritmos devolvieron los mismos 15 términos, en el mismo orden y con las mismas frecuencias (Tabla 27). Encabeza la lista «ai», con 86 apariciones, seguido de «education» (57) y «research» (46). La coincidencia va más allá del top 15: las 950 posiciones del ranking completo son idénticas en los tres resultados.

Tabla 27

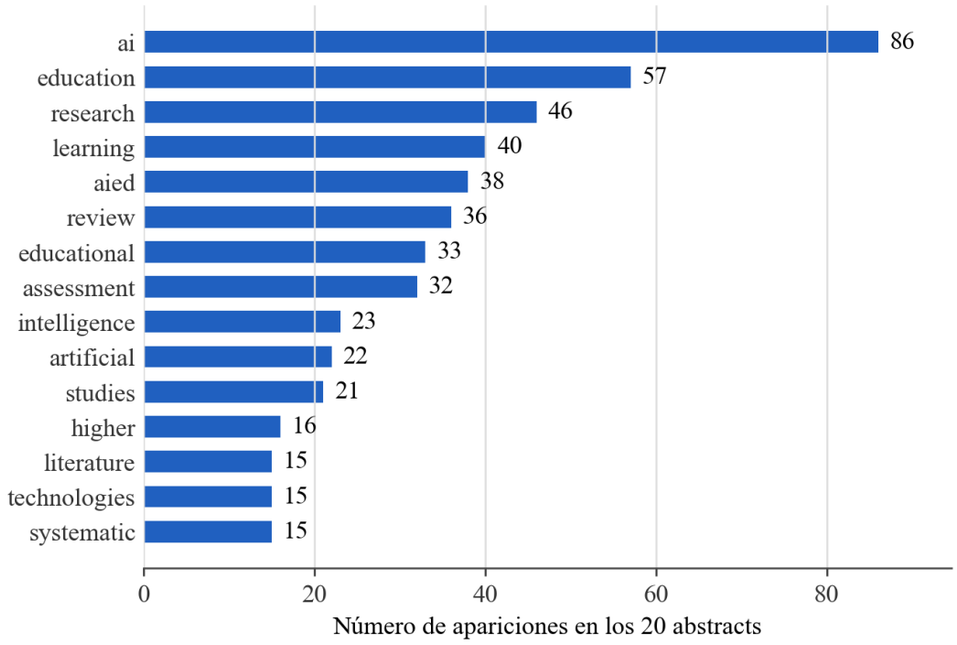
Los 15 términos más frecuentes del corpus según cada algoritmo

Posición	Counting Sort	Radix Sort	Bucket Sort
1	ai (86)	ai (86)	ai (86)
2	education (57)	education (57)	education (57)
3	research (46)	research (46)	research (46)
4	learning (40)	learning (40)	learning (40)
5	aied (38)	aied (38)	aied (38)
6	review (36)	review (36)	review (36)
7	educational (33)	educational (33)	educational (33)
8	assessment (32)	assessment (32)	assessment (32)
9	intelligence (23)	intelligence (23)	intelligence (23)
10	artificial (22)	artificial (22)	artificial (22)
11	studies (21)	studies (21)	studies (21)
12	higher (16)	higher (16)	higher (16)
13	literature (15)	literature (15)	literature (15)
14	technologies (15)	technologies (15)	technologies (15)
15	systematic (15)	systematic (15)	systematic (15)

Nota. Entre paréntesis, número de apariciones del término en los 20 abstracts.

Figura 14

Frecuencia de los 15 términos más frecuentes del corpus



Nota. El resultado es idéntico para Counting Sort, Radix Sort y Bucket Sort. Elaboración propia.

Cuatro términos empatan con 15 apariciones: literature, technologies, systematic y study. Study queda en la posición 16 porque aparece por primera vez después de los otros tres, unas palabras más adelante en el artículo 2. Como los tres algoritmos son estables, el empate se resolvió igual en todos.

Para descartar errores en el conteo lo repetimos con un programa independiente escrito en Python, que está en el repositorio como verificacion/verificar_conteo.py. El programa no reutiliza el código Java: vuelve a escribir las reglas de preprocesamiento (normalización NFKD, minúsculas, tokenización por caracteres que no son letras, stopwords y tokens de una letra) usando solo la biblioteca estándar (Python Software Foundation, 2024), cuenta los términos de los 20 abstracts y los ordena con sorted(), que es un ordenamiento por comparación y estable (Timsort). Obtuvo las mismas cifras de la Tabla 21 y el mismo ranking que los tres algoritmos en las 950 posiciones, incluido el orden de los empates. La salida completa está en el Anexo C.

El ranking tiene sentido para el tema del corpus. «ai» es la sigla y se cuenta aparte de «artificial» e «intelligence», mientras que «aied» corresponde a la sigla AIEd (Artificial

Intelligence in Education). Unir siglas y formas completas habría exigido un diccionario, algo que este requerimiento no pedía.

8. Requerimiento 4: despliegue y documentación

La aplicación se distribuye como un único JAR ejecutable (java -jar bibliometria.jar), que incluye el servidor web, la interfaz y Apache PDFBox. Para desplegarla en la nube el repositorio incluye un Dockerfile de dos etapas: la primera compila el proyecto con Maven y ejecuta las 52 pruebas unitarias, y la segunda deja solo el entorno de ejecución de Java, el JAR y la carpeta del corpus, con los abstracts, los metadatos y los embeddings ya generados. El servidor escucha en el puerto de la variable de entorno PORT, la convención de servicios como Render o Railway.

La aplicación está desplegada en Render como servicio web con entorno Docker, en el plan gratuito, y se puede usar en la dirección:

https://bibliometria-proyecto-final.onrender.com

Render construye la imagen a partir del repositorio público del proyecto en GitHub, que contiene el código, el corpus y este documento:

https://github.com/NicolasBuitragoMartinez/bibliometria-proyecto-final

En el plan gratuito el servidor se detiene tras 15 minutos sin uso, así que la primera consulta después de una pausa tarda unos 50 segundos mientras arranca; las siguientes responden con normalidad. La interfaz también acepta enlaces directos a los casos de estudio: al final de la dirección se puede agregar #similitud=11,16 (sección 4), #pasos=analysis,18,426 (sección 6.6) o #ranking (capítulo 7).

La documentación técnica la forman este documento, el README del repositorio (requisitos, ejecución, despliegue y estructura) y los comentarios de cada clase, que explican el algoritmo, su recurrencia y su complejidad. Las pruebas unitarias verifican los algoritmos con casos conocidos: la distancia de Levenshtein entre «kitten» y «sitting», la función de prefijos de «ababaca», los tres criterios de enlace sobre cuatro puntos en una recta, la

correlación cofenética y la silueta calculadas a mano, y la comparación de KMP, Boyer-Moore y Rabin-Karp (también con q = 13) con la fuerza bruta en 300 textos aleatorios. También comprueban que el paso a paso de la búsqueda recorre el texto igual que cada algoritmo, comparan Counting, Radix y Bucket Sort con el ordenamiento estable de Java y verifican la extracción del año con el formato de cita de cada editorial.

9. Conclusiones

Los seis algoritmos de similitud miden cosas distintas, y el caso de estudio lo mostró con claridad: dos artículos sobre el mismo tema, la IA generativa y la evaluación, obtuvieron similitudes entre 0,07 y 0,93 según el algoritmo. Las medidas de edición y Jaccard dependen de que se repitan las mismas palabras o letras y no reconocen el parecido temático; el coseno TF-IDF sí lo reconoce porque da más peso a las palabras compartidas que son poco frecuentes en el corpus; y MiniLM, que interpreta cada palabra en su contexto, es el que mejor refleja la relación entre los textos. El valor de una similitud no es comparable entre algoritmos: hay que interpretarlo frente a la distribución de todos los pares, como mostró el caso de GloVe.

En el agrupamiento jerárquico, average linkage produjo los dendrogramas más fieles a las distancias reales con las tres representaciones (CCC entre 0,80 y 0,83), por lo que es el criterio más coherente para este corpus. La silueta mostró además que la representación pesa más que el criterio, porque con TF-IDF no hay grupos bien separados y con embeddings sí aparecen grupos temáticos, y que una silueta alta puede deberse a separar artículos atípicos más que a encontrar temas.

En la búsqueda de patrones, las mediciones confirmaron la complejidad lineal de los tres algoritmos y el comportamiento sublineal de Boyer-Moore en texto natural, que fue el más rápido. También mostraron que el número de comparaciones no basta para predecir el tiempo: Rabin-Karp compara menos caracteres que los otros dos, pero es el más lento porque domina el costo de cada paso del hash. Su versión multipatrón muestra dónde sí conviene: con diez patrones de la misma longitud recorrió el texto una sola vez y fue el más rápido (3,99 ms frente a 5,92 ms de Boyer-Moore).

En el ranking de términos se cumplen las condiciones que hacen lineales a Counting, Radix y Bucket Sort: la frecuencia es un entero acotado (k = 86) y menor que el número de términos (n = 950). Los tres produjeron el mismo ranking de 950 términos, confirmado por un programa independiente en Python, y Counting Sort fue el más rápido porque solo recorre arreglos de enteros en orden. El experimento de jerarquía de memoria mostró, además, que con exactamente las mismas operaciones el costo por término puede ser hasta 7,0 veces mayor si los objetos están dispersos en la memoria: la notación O predice cómo crece el número de operaciones, pero no lo que cuesta cada una.

Implementar los algoritmos desde cero nos permitió seguir su funcionamiento paso a paso con los datos del corpus y relacionar cada resultado con su fórmula. Los modelos de IA, usados solo para vectorizar los textos, se integraron como una representación más sobre la que se aplican las mismas métricas, lo que facilitó compararlos de manera directa con los enfoques clásicos.

10. Declaración de uso de inteligencia artificial

Declaramos que en este proyecto usamos Claude (Anthropic, 2026), un asistente de inteligencia artificial generativa, como herramienta de apoyo, tal como lo permite el enunciado del curso. Lo usamos para pedir asesoría sobre los algoritmos y su análisis, comparar alternativas de implementación y de diseño de los experimentos, investigar técnicas y bibliotecas (por ejemplo, los modelos de embeddings y la forma de medir tiempos en la JVM), proponer y escribir mejoras y actualizaciones al código y a la interfaz a partir de nuestro avance, atender las observaciones del docente al seguimiento, revisar la redacción de este documento y acompañarnos en el despliegue en Render.

El diseño de la solución, la elección de los casos de estudio y la interpretación de los resultados son nuestros. Revisamos, ejecutamos y probamos todo el código entregado; verificamos los algoritmos con las pruebas unitarias, con los cálculos a mano de los casos de estudio y con un programa independiente en Python, y las cifras del documento salen de nuestras propias ejecuciones, cuyos archivos están en la carpeta resultados del repositorio.

Los prompts utilizados, su propósito y la forma en que revisamos cada respuesta se detallan en el documento de declaración de uso de IA que acompaña la entrega.

11. Referencias bibliográficas

Algoritmo de ordenamiento RadixSort [Material de clase]. (s. f.). Universidad del Quindío.

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai

Apache Software Foundation. (2025). *Apache PDFBox* (Versión 3.0.5) [Software]. https://pdfbox.apache.org/

Bahroun, Z., Anane, C., Ahmed, V., y Zacca, A. (2023). Transforming education: A comprehensive review of generative artificial intelligence in educational settings through bibliometric and content analysis. *Sustainability, 15*(17), Artículo 12983. https://doi.org/10.3390/su151712983

Boyer, R. S., y Moore, J. S. (1977). A fast string searching algorithm. *Communications of the ACM, 20*(10), 762–772. https://doi.org/10.1145/359842.359859

Cardona Torres, S. A. (s. f.). *Análisis de métodos de ordenamiento* [Material de clase, núcleo temático 4]. Universidad del Quindío.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Drepper, U. (2007). *What every programmer should know about memory*. Red Hat. https://people.freebsd.org/~lstewart/articles/cpumemory.pdf

GeeksforGeeks. (s. f.). *Sorting algorithms*. Recuperado el 25 de octubre de 2026, de https://www.geeksforgeeks.org/dsa/sorting-algorithms/

Hennessy, J. L., y Patterson, D. A. (2019). *Computer architecture: A quantitative approach* (6.ª ed.). Morgan Kaufmann.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Hyndman, R. J., y Fan, Y. (1996). Sample quantiles in statistical packages. *The American Statistician, 50*(4), 361–365. https://doi.org/10.1080/00031305.1996.10473566

Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et du Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles, 37*, 547–579.

Karp, R. M., y Rabin, M. O. (1987). Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development, 31*(2), 249–260. https://doi.org/10.1147/rd.312.0249

Knuth, D. E., Morris, J. H., y Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM Journal on Computing, 6*(2), 323–350. https://doi.org/10.1137/0206024

Lance, G. N., y Williams, W. T. (1967). A general theory of classificatory sorting strategies: 1. Hierarchical systems. *The Computer Journal, 9*(4), 373–380. https://doi.org/10.1093/comjnl/9.4.373

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady, 10*(8), 707–710.

Lye, C. Y., y Lim, L. (2024). Generative artificial intelligence in tertiary education: Assessment redesign principles and considerations. *Education Sciences, 14*(6), Artículo 569. https://doi.org/10.3390/educsci14060569

Manning, C. D., Raghavan, P., y Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Needleman, S. B., y Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology, 48*(3), 443–453. https://doi.org/10.1016/0022-2836(70)90057-4

Pennington, J., Socher, R., y Manning, C. D. (2014). GloVe: Global vectors for word representation. En *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing* (pp. 1532–1543). https://doi.org/10.3115/v1/D14-1162

Python Software Foundation. (2024). *Python* (versión 3.11) [Software]. https://www.python.org/

Qdrant. (2024). *FastEmbed* [Software]. https://github.com/qdrant/fastembed

Reimers, N., y Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. En *Proceedings of the 2019 Conference on Empirical Methods in*

Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (pp. 3982–3992). https://doi.org/10.18653/v1/D19-1410

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Salton, G., y Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Sokal, R. R., y Rohlf, F. J. (1962). The comparison of dendrograms by objective methods. *Taxon, 11*(2), 33–40. https://doi.org/10.2307/1217208

Spearman, C. (1904). The proof and measurement of association between two things. *The American Journal of Psychology, 15*(1), 72–101. https://doi.org/10.2307/1412159

Universidad del Quindío. (2026). *Análisis de algoritmos en el contexto de la bibliometría: Descripción final del proyecto* [Enunciado de la actividad]. Programa de Ingeniería de Sistemas y Computación.

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., y Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. En *Advances in Neural Information Processing Systems 33* (pp. 5776–5788).

Xia, Q., Weng, X., Ouyang, F., Lin, T. J., y Chiu, T. K. F. (2024). A scoping review on how generative artificial intelligence transforms assessment in higher education. *International Journal of Educational Technology in Higher Education, 21*, Artículo 40. https://doi.org/10.1186/s41239-024-00468-z

Anexo A. Título, autores y año extraídos de los PDF

La Tabla A1 lista el título, los autores y el año que la aplicación extrajo automáticamente de cada PDF (sección 3.1), tal como quedan en la carpeta corpus/metadatos.

Tabla A1

Título, autores y año de los 20 artículos

N.°	Título	Autores	Año
1	Artificial intelligence literacy in higher and adult education: A scoping literature review	Matthias Carl Laupichler, Alexandra Aster, Jana Schirch, Tobias Raupach	2022
2	Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education	Thomas K.F. Chiu, Qi Xia, Xinyan Zhou, Ching Sing Chai, Miaoting Cheng	2023
3	Myths, mis- and preconceptions of artificial intelligence: A review of the literature	Arne Bewersdorff, Xiaoming Zhai, Jessica Roberts, Claudia Nerdel	2023
4	Opportunities of artificial intelligence for supporting complex problem-solving: Findings from a scoping review	Srecko Joksimovic, Dirk Ifenthaler, Rebecca Marrone, Maarten De Laat, George Siemens	2023
5	Application and theory gaps during the rise of Artificial Intelligence in Education	Xieling Chen, Haoran Xie, Di Zou, Gwo-Jen Hwang	2020
6	A multi-perspective study on Artificial Intelligence in Education: grants, conferences, journals, software tools, institutions, and researchers	Xieling Chen, Haoran Xie, Gwo-Jen Hwang	2020
7	Human-centred learning analytics and AI in education: A systematic literature review	Riordan Alfredo, Vanessa Echeverria, Yueqiao Jin, Lixiang Yan, Zachari Swiecki, Dragan Gašević, Roberto Martinez-Maldonado	2024
8	A systematic review of artificial intelligence techniques for collaborative learning over the past two decades	Seng Chee Tan, Alwyn Vwen Yen Lee, Min Lee	2022
9	AI technologies for education: Recent research & future directions	Ke Zhang, Ayse Begum Aslan	2021
10	Educational applications of artificial intelligence in simulation-based learning: A systematic mapping review	Chih-Pu Dai, Fengfeng Ke	2022
11	A scoping review on how generative artificial intelligence transform assessment in higher education	Qi Xia, Xiaojing Weng, Fan Ouyang, Tzung Jin Lin, Thomas K.F. Chiu	2024
12	A meta systematic review of artificial intelligence in higher education: a call for increased ethics, collaboration, and rigour	Melissa Bond, Hassan Khosravi, Maarten De Laat, Nina Bergdahl, Violeta Negrea, Emily Oxley, Phuong Pham, Sin Wang Chong, George Siemens	2024
13	Artificial intelligence for education and its emphasis on assessment and adversity quotient: a review	Jyoti Mudkanna Gavhane, Reena Pagare	2024

N.°	Título	Autores	Año
14	A comprehensive exploration of artificial intelligence competence frameworks for educators: A critical review	Tamar Mikeladze, Paulien C. Meijer, Roald P. Verhoeff	2024
15	A Systematic Review of Generative AI for Teaching and Learning Practice	Bayode Ogunleye, Kudirat Ibilola Zakariyyah, Oluwaseun Ajao, Olakunle Olayinka, Hemlata Sharma	2024
16	Generative Artificial Intelligence in Tertiary Education: Assessment Redesign Principles and Considerations	Che Yee Lye, Lyndon Lim	2024
17	A Systematic Review of AI-Driven Educational Assessment in STEM Education	Fan Ouyang, Tuan Anh Dinh, Weiqi Xu	2023
18	Transforming Education: A Comprehensive Review of Generative Artificial Intelligence in Educational Settings through Bibliometric and Content Analysis	Zied Bahroun, Chiraz Anane, Vian Ahmed, Andrew Zacca	2023
19	What Is the Impact of ChatGPT on Education? A Rapid Review of the Literature	Chung Kwan Lo	2023
20	Systematic review of research on artificial intelligence applications in higher education – where are the educators?	Olaf Zawacki-Richter, Victoria I. Marín, Melissa Bond, Franziska Gouverneur	2019

Anexo B. Código de los algoritmos de ordenamiento

Este anexo reproduce las clases de los tres algoritmos del capítulo 7 y la cola enlazada que usa Radix Sort, tal como están en el paquete ranking.ordenamiento del repositorio.

Counting Sort (CountingSort.java)

```
public final class CountingSort implements AlgoritmoOrdenamiento {

    @Override
    public String nombre() {
        return "Counting Sort";
    }

    @Override
    public Termino[] ordenar(Termino[] terminos) {
        int n = terminos.length;
        Termino[] salida = new Termino[n];
        if (n == 0) {
            return salida;
        }
        int min = AlgoritmoOrdenamiento.frecuenciaMinima(terminos);
        int max = AlgoritmoOrdenamiento.frecuenciaMaxima(terminos);

        // 1. Histograma: cuantos terminos tienen cada frecuencia
        int[] conteo = new int[max - min + 1];
        for (Termino t : terminos) {
            conteo[t.frecuencia() - min]++;
        }

        // 2. Posicion inicial de cada frecuencia en la salida de la mayor a la menor
        int posicion = 0;
        for (int v = conteo.length - 1; v >= 0; v--) {
            int cantidad = conteo[v];
            conteo[v] = posicion;
            posicion += cantidad;
        }

        // 3. Colocacion recorriendo la entrada en orden
        for (Termino t : terminos) {
            salida[conteo[t.frecuencia() - min]++] = t;
        }
        return salida;
    }
}
```

Radix Sort (RadixSort.java)

```
public final class RadixSort implements AlgoritmoOrdenamiento {

    private static final int BASE = 10;

    @Override
    public String nombre() {
        return "Radix Sort";
    }

    @Override
    public Termino[] ordenar(Termino[] terminos) {
        Termino[] arreglo = terminos.clone();
        int numeroDigitos =
String.valueOf(AlgoritmoOrdenamiento.frecuenciaMaxima(terminos)).length();

        @SuppressWarnings({"unchecked", "rawtypes"})
        ColaEnlazada<Termino>[] colas = new ColaEnlazada[BASE];
        for (int d = 0; d < BASE; d++) {
            colas[d] = new ColaEnlazada<>();
        }

        int divisor = 1;
        for (int i = 1; i <= numeroDigitos; i++) {
            for (Termino t : arreglo) {
```

```
                colas[(t.frecuencia() / divisor) % BASE].encolar(t);
            }
            int posicion = 0;
            for (int d = BASE - 1; d >= 0; d--) {
                while (!colas[d].estaVacia()) {
                    arreglo[posicion++] = colas[d].decolar();
                }
            }
            divisor *= BASE;
        }
        return arreglo;
    }
}
```

Bucket Sort (BucketSort.java)

```
public final class BucketSort implements AlgoritmoOrdenamiento {

    @Override
    public String nombre() {
        return "Bucket Sort";
    }

    @Override
    public Termino[] ordenar(Termino[] terminos) {
        int n = terminos.length;
        Termino[] salida = new Termino[n];
        if (n == 0) {
            return salida;
        }
        int min = AlgoritmoOrdenamiento.frecuenciaMinima(terminos);
        long rango = (long) AlgoritmoOrdenamiento.frecuenciaMaxima(terminos) - min + 1;

        // 1. Distribucion en cubetas
        @SuppressWarnings({"unchecked", "rawtypes"})
        List<Termino>[] cubetas = new List[n];
        for (Termino t : terminos) {
            int indice = (int) ((t.frecuencia() - min) * (long) n / rango);
            if (cubetas[indice] == null) {
                cubetas[indice] = new ArrayList<>();
            }
            cubetas[indice].add(t);
        }

        // 2. Ordenar cada cubeta y concatenar desde la cubeta de frecuencias mas altas
        int posicion = 0;
        for (int i = n - 1; i >= 0; i--) {
            List<Termino> cubeta = cubetas[i];
            if (cubeta != null) {
                insercionDescendente(cubeta);
                for (Termino t : cubeta) {
                    salida[posicion++] = t;
                }
            }
        }
        return salida;
    }

    private static void insercionDescendente(List<Termino> cubeta) {
        for (int i = 1; i < cubeta.size(); i++) {
            Termino actual = cubeta.get(i);
            int j = i - 1;
            while (j >= 0 && cubeta.get(j).frecuencia() < actual.frecuencia()) {
                cubeta.set(j + 1, cubeta.get(j));
                j--;
            }
            cubeta.set(j + 1, actual);
        }
    }
}
```

Cola enlazada usada por Radix Sort (ColaEnlazada.java)

```
final class ColaEnlazada<T> {

    private static final class Nodo<T> {
        private final T valor;
        private Nodo<T> siguiente;
```

```
        Nodo(T valor) {
            this.valor = valor;
        }
    }

    private Nodo<T> inicio;
    private Nodo<T> fin;

    void encolar(T valor) {
        Nodo<T> nuevo = new Nodo<>(valor);
        if (inicio == null) {
            inicio = nuevo;
        } else {
            fin.siguiente = nuevo;
        }
        fin = nuevo;
    }

    T decolar() {
        T valor = inicio.valor;
        inicio = inicio.siguiente;
        if (inicio == null) {
            fin = null;
        }
        return valor;
    }

    boolean estaVacia() {
        return inicio == null;
    }
}
```

Anexo C. Verificación independiente en Python

El programa verificacion/verificar_conteo.py se ejecuta desde la carpeta del proyecto con python verificacion/verificar_conteo.py y solo necesita Python 3.8 o superior. Esta es su salida sobre el corpus:

```
Verificación independiente en Python
  Artículos                              20
  Palabras en los abstracts              4.007
  Términos tras el preprocesamiento      2.407
  Términos distintos (n)                 950
  Frecuencia máxima / mínima             86 / 1
  Rango de claves k = máx - mín + 1      86
  Términos que aparecen una sola vez     575 (60,5 %)


Top 15 (sorted de Python):
   1. ai (86)
   2. education (57)
   3. research (46)
   4. learning (40)
   5. aied (38)
   6. review (36)
   7. educational (33)
   8. assessment (32)
   9. intelligence (23)
  10. artificial (22)
  11. studies (21)
  12. higher (16)
  13. literature (15)
  14. technologies (15)
  15. systematic (15)


Comparación con resultados/ranking/ranking_completo.csv (aplicación Java):
  Términos en Python / en Java           950 / 950
  Mismos términos y frecuencias          SÍ
  Mismo orden en las 950 posiciones      SÍ
```

VIGILADA MINEDUCACIÓN



PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Tel: (57) 6 735 9300 Ext 355

Carrera 15 Calle 12 Norte

Armenia, Quindío – Colombia

ingesis@uniquindio.edu.co